GAN 합성데이터 기반 유방암 생존분석

김서윤[1)], 정선우[1)]

1. 서론
  1.1 연구 배경

2. 유방암 데이터
  2.1 유방암 데이터 출처
  2.2 데이터 전처리

3. 유방암 생존분석
  3.1 Kaplan-Meier 생존곡선과 누적위험함수
  3.2 Log-rank Test
  3.3 Cox 비례위험모형
    3.3.1 단변량 Cox 비례위험모형
    3.3.2 공변량이 포함되지 않은 경우
    3.3.3 모든 공변량을 포함한 경우
    3.3.4 후진 제거법을 이용한 변수 선택
  3.4 최종모형
    3.4.1 최종모형 변수의 상호작용 효과 검정
    3.4.2 비례위험가정 검정 및 잔차 진단
      3.4.2.1 Schoenfeld 잔차 기반 비례위험가정 검정
      3.4.2.2 로그-로그 생존곡선을 이용한 시각적 진단
      3.4.2.3 누적위험함수 기반 진단
      3.4.2.4 Martingale 잔차 및 Deviance 잔차 분석

4. 결론 및 한계
  4.1 분석결과 요약
  4.2 변수 선택 결과 및 최종모형
  4.3 상호작용 분석 결과
  4.4 모형 적합성 및 비례위험 가정 검정
  4.5 한계점
  4.6 최종 결론

참고문헌
부록

의학데이터에 대한 생존분석실무(캡스톤디자인)

요약

본 연구는 국가암데이터센터의 유방암 임상 데이터를 기반으로 인공지능 생성모델 알고리즘을 통해 생성된 합성 데이터를 기반으로 생존분석을 수행하였다. Kaplan-Meier 추정, 로그-순위 검정, Cox 비례위험모형을 적용하여 예후 인자 탐색과 모형 선택 과정을 단계적인 분석과정을 거쳤다. 단변량 Cox 모형에서는 조직학적 진단 중 lobular carcinoma 만이 사망 위험 감소와 유의한 관련성을 보였으며, BMI와 BRCA 병원성 변이는 경계선 수준의 효과를 나타냈다. 모든 공변량을 포함한 풀모형과 AIC 기반 변수 선택 결과에서도 전반적인 예측 성능은 제한적인 수준(C-index 약 0.52-0.53)에 머물렀다.

분석 결과, 합성 데이터 환경에서 전체 생존 추세는 일정 수준 재현되나, 개별 임상 예후 인자에 따른 위험 구조는 충분히 반영되지 않을 수 있음을 보여준다. 이러한 결과는 합성 데이터 기반 생존 분석에서 통계적 유의성 해석과 모형 선택 시 주의가 필요함을 시사한다.

주요용어 : GAN 합성 데이터, Kaplan_Meier 생존곡선, 누적위험함수, 유방암

1. 서론

1.1 연구 배경

유방암은 국내 여성에서 가장 높은 발생률을 보이는 악성 종양 중 하나로, 환자의 생존 예후는 병기(Stage), 호르몬 수용체 상태(ER/PR), 림프절 전이 여부, 조직학적 진단, 치료 방식 등 다양한 임상·병리학적 요인에 의해 영향을 받는다. 이러한 예후 인자들은 임상 의사결정과 치료 전략 수립의 핵심 근거가 되며, 이를 정확히 규명하기 위해서는 대규모의 고품질 실제 임상데이터가 필요하다. 그러나 개인정보 보호 규제 강화로 인해 실제 임상데이터의 활용이 점차 제한되고 있으며, 이를 대체하거나 보완하기 위한 방안으로 합성 의료데이터가 주목받고 있다. 특히 Generative Adversarial Networks(GAN)는 원본 데이터의 통계적 분포를 학습하여 유사한 특성을 갖는 데이터를 생성할 수 있어 민감 정보 보호와 데이터 활용 간의 균형을 가능하게 한다.

하지만, GAN 기반 합성 데이터가 임상 예후 인자와 생존 결과 간의 관계와 같은 구조적·인과적 특성을 실제 데이터 수준으로 재현할 수 있는지에 대해서는 충분한 검증이 이루어지지 않았다. 특히 생존 분석은 다변량 공변량과 시간 의존적 위험 구조를 포함하므로, 합성데이터의 임상적 타당성을 평가하는 데 중요한 기준이 된다. 본 연구의 목적은 유방암 환자의 GAN 합성데이터를 대상으로 생존 분석을 수행하여, 주요 임상 예후 인자와 생존 결과 간의 관계가 실제 임상에서 관찰되는 패턴을 얼마나 충실히 반영하는지를 평가하고자 한다.

2. 유방암 데이터

2.1 유방암 데이터 출처

본 연구에서 사용한 데이터는 국가암데이터센터에서 제공한 유방암 환자 합성데이터로, 5년 이내 원내 사망 정보, 기본정보, 병기STAGE, 건강정보, 신체계측정보, 유전체 정보(BRACA1, BRACA2), 면역병리, 수술정보, 항암제치료, 호르몬치료, 방사선치료 정보를 포함하고 있다.
해당 합성데이터는 유방암 라이브러리 기반의 실제 임상 데이터를 인공지능 생성모델 알고리즘에 적용하여 생성된 것으로 총 59개의 변수로 구성되어있다. 본 연구에서는 이 합성데이터

를 활용하여 생존 기간과 사망 여부를 종속 변수로 설정하고, 임상·병리학적 예후 인자들을 공변량으로 포함한 생존분석을 수행하였다.

2.2 데이터 전처리

본 연구에 사용된 유방암 합성데이터를 생존분석에 직접 활용하기 위해서는 변수 간 중복과 논리적 불일치를 해소하는 전처리 과정이 필요하였다. 특히 비표준 결측값의 광범위한 사용, 동일 임상 개념의 중복 변수화, 희귀 범주의 존재, 생식 관련 변수의 논리적 모순, 과도하게 세분화된 치료 변수 등이 확인되었다. 이에 따라 본 연구에서는 임상적 해석 가능성과 통계적 안정성을 확보하기 위해 전처리 원칙을 다음과 같이 설정하였다. 첫째, 임상적으로 의미 있는 범주는 최대한 보존하되, 해석이 불가능하거나 모순되는 값은 제거하거나 재구성하였다. 둘째, 동일한 임상 개념을 여러 변수로 표현한 경우 단일 변수로 통합하였다. 셋째, 분석 목적에 기여하지 않는 중복된 변수는 제외하였다. 이러한 기준에 따라 총 57개의 원본 변수를 29개의 분석 변수로 축소하였다.

결측치 처리에서는 호르몬 수용체(ER, PR, AR) 변수의 비표준 코드 중 임상적으로 “미검사”를 의미하는 값은 하나의 범주로 유지하였으며, 그 외 변수의 결측값은 관측치를 제거하거나 임상적 타당성을 고려한 대체 방식을 적용하였다. 생식 관련 변수에서는 초경, 폐경, 출산 변수들은 불가능한 값은 중앙값으로 대치하고 폐경 여부 및 출산 여부를 나타내는 이진 변수를 새로 생성함으로써 변수 구조를 단순화하였다.

호르몬 수용체 정보는 정제된 ER 및 PR 조합을 기반으로 통합 호르몬 상태 변수(Hormone status)를 구성하였다. 조직학적 진단은 중복 기재된 여러 변수들을 하나의 Diagnosis 변수로 통합하였으며, 희귀하거나 분류가 불명확한 경우에는 기타 범주로 묶어 분석의 안정성을 확보하였다.

TNM 병기 정보는 개별 병기 수준으로 재구성한 뒤 AJCC 기준에 따라 통합 병기(Stage)를 산출하였다. 이를 통해 병기 정보의 중복을 제거하고, 생존 분석에 활용 가능한 단일 병기 변수로 정리하였다. BRCA 유전자 정보 역시 여러 원본 변수를 하나의 BRCA status 변수로 통합하여 병원성 변이, 변이 불확실, 음성 및 미상 범주로 재정의하였다.

연령과 체질량지수(BMI)는 임상적 의미를 반영한 범주형 변수로 변환하였으며, 치료 관련 변수는 수술, 항암치료, 방사선치료, 호르몬치료의 조합을 기반으로 단일 치료 변수로 통합하였다. 또한 분석에 기여하지 않는 원본 병기 더미 변수, 중복된 호르몬 및 BRCA 변수, 신체계측 원변수 등은 최종 분석 단계에서 제외하였다.

<표 2.2> 전처리 한 유방암 데이터 대한 변수설명표

변수명	설명	값의 의미	변수 유형
AGE	진단 시 연령	예: 45세, 62세	연속형
Smoke	흡연 여부	1=흡연, 0=비흡연	이진형
Age_of_Menarch	초경 연령	예: 13세, 14세	연속형
Age_of_Menopause	폐경 연령	예: 49세, 52세	연속형
Age_at_first_birth	첫 출산 연령	예: 28세, 32세	연속형
Number_of_births	출산 횟수	예: 0, 1, 2	연속형
Chemotherapy	항암화학요법 여부	1=시행, 0=미시행	이진형
Hormone_therapy	호르몬 치료 여부	1=시행, 0=미시행	이진형
Radiation_Therapy	방사선 치료 여부	1=시행, 0=미시행	이진형
Death	사망 여부	1=사망, 0=생존	이진형
Survival_period	생존 기간(일)	예: 120, 342	연속형

Menopause_status	폐경 상태	1=폐경, 0=비폐경	이진형
Has_birth	출산 여부	1=출산 경험 있음, 0=없음	이진형
ER_clean	에스트로겐 수용체	1=양성, 2=음성, 99=미검사	범주형
PR_clean	프로게스테론 수용체	1=양성, 2=음성, 99=미검사	범주형
AR_clean	안드로겐 수용체	1=양성, 2=음성	이진형
Hormone_status	호르몬 종합 상태	ER·PR 조합(예: 1_1=ER+/PR+)	범주형
Diagnosis	조직학적 진단	ductal, lobular 등	범주형
T_stage	원발 종양 병기	T1~T4	범주형
N_stage	림프절 병기	N0~N3	범주형
M_stage	전이 병기	M0, M1	범주형
Stage	TNM 통합 병기	0기~4기	범주형
BRCA_status	BRCA 병원성 변이	Unknown, VUS 등	범주형
BMI	체질량 지수	예: 21.5	연속형
BMI_group	BMI 그룹	Under / Normal / Over / Obese	범주형
AGE_group	연령 그룹	Young / Middle / Old	범주형
Surgery_type	수술 유형	0=없음, 1=BCS 등	이진형
treat	치료 조합	Surgery+Chemo 등	범주형
Alcohol	음주 여부	1=음주, 0=비음주	이진형

3. 유방암 생존분석

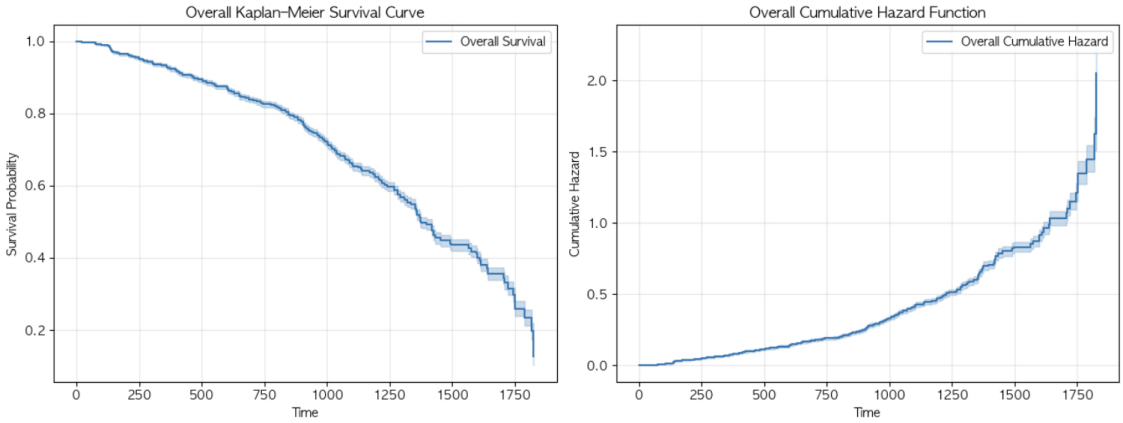
3.1 Kaplan-Meier 생존곡선과 누적위험함수

생존분석에서는 시간에 따른 생존 확률을 추정하고 집단 간 생존 경험의 차이를 비교하기 위해 Kaplan-Meier 생존곡선과 누적위험함수(cumulative hazard function)가 활용된다. Kaplan-Meier 곡선은 사건 발생 시점 정보를 바탕으로 비모수적으로 생존함수를 추정하여 집단 간 생존율 비교에 중요한 근거를 제공하며, 누적위험함수는 시간 경과에 따라 위험이 어떻게 누적되는지를 보여줌으로써 질병의 진행 양상을 해석하는 데 유용하다. 이러한 이유로 본 연구에서도 Kaplan-Meier 생존곡선과 누적위험함수를 활용하여 합성 데이터가 실제 임상적 생존 패턴을 얼마나 재현하는지 평가하였으며, 생존함수는 다음과 같다.

$$\hat{S}(t) = \Pi_{t_i \le t}(1 - \frac{d_i}{n_i})$$

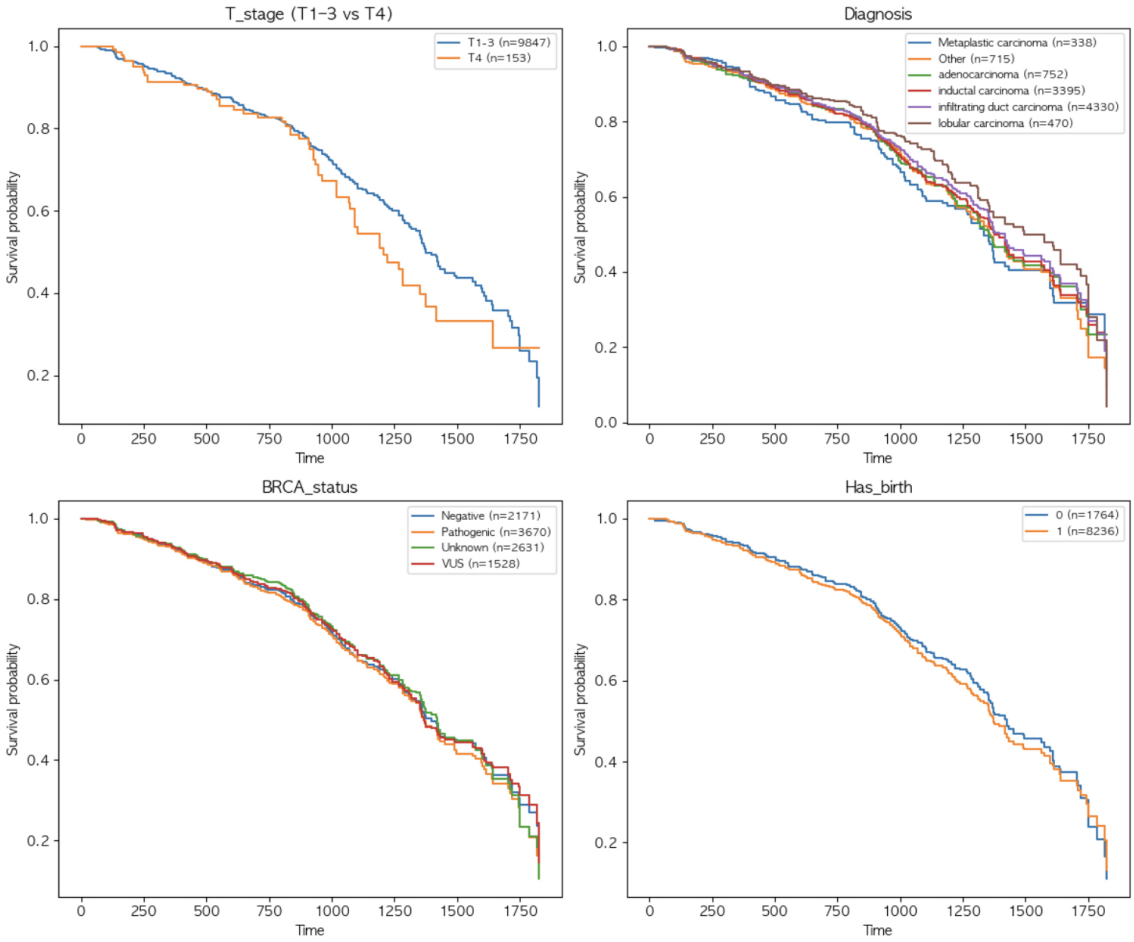
<그림3.1>은 전체 환자를 대상으로 Kaplan-Meier 생존곡선과 누적위험함이다. Kaplan-Meier 추정 결과, 1년 생존율은 92.7%, 2년 생존율은 83.3%, 3년 생존율은 66.4%로 나타났다. 이러한 생존율 수준은 실제 유방암 환자 장기 생존 추세와 대체로 유사한 범위 안에 있으며, 누적위험률 또한 시간에 따라 점진적으로 증가하는 전형적인 형태를 보였다. 즉, 전체 차원의 생존곡선만을 기준으로 보면 GAN 모델이 기본적인 생존 함수의 감소 구조는 어느 정도 모사하고 있는 것으로 판단된다. 그러나 개별 임상변수가 반영하는 위험 구조를 평가하기 위

해서는 변수를 구분한 생존곡선 비교가 필수적이며, 다음 분석에서 이러한 세부 패턴의 재현 여부를 확인하였다.



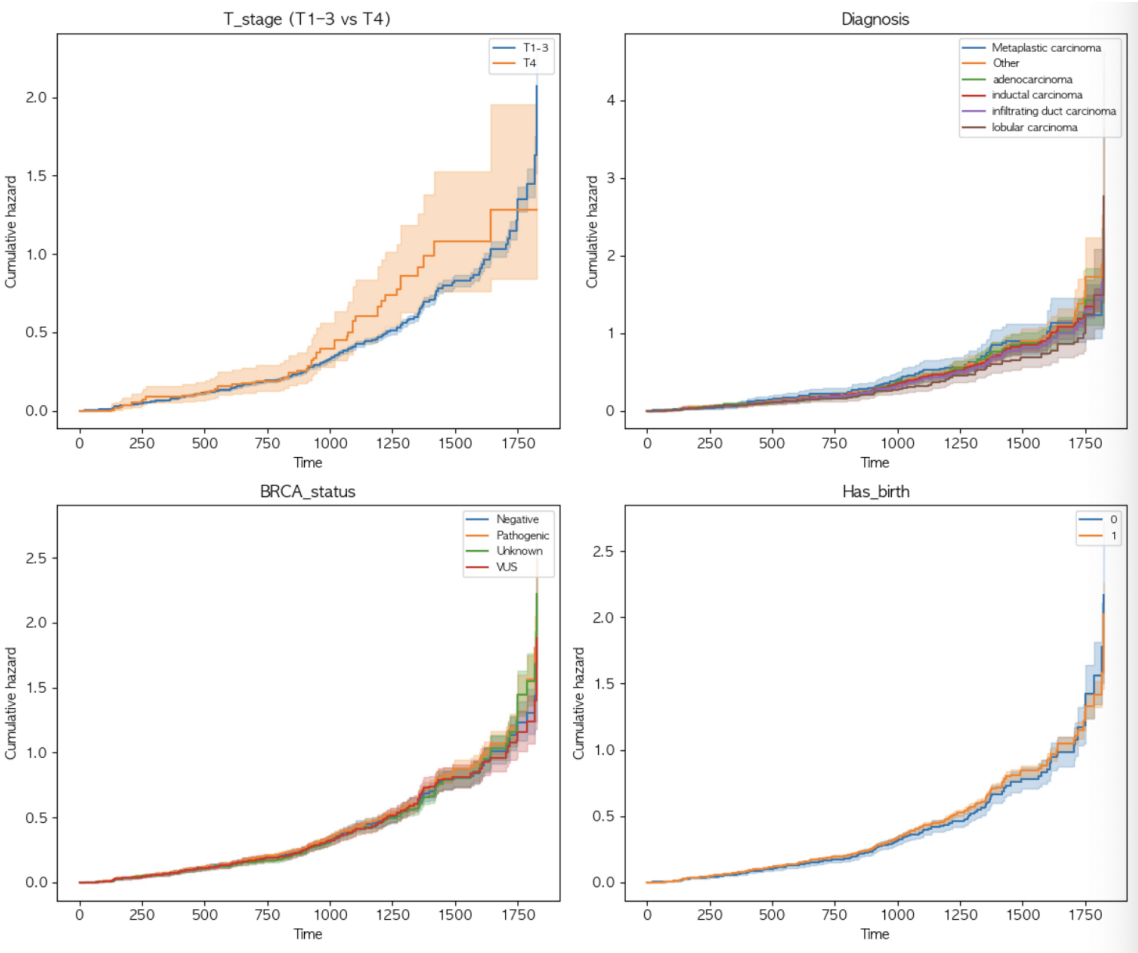
<그림 3.1> 전체 유방암 환자에 대한 KM곡선과 누적위험함수

<그림 3.2>는 주요 임상 변수에 따라 구분한 Kaplan-Meier 생존곡선이다. 먼저 종양 크기에 따른 T 병기 비교에서는 T1-3군과 T4군 간의 생존곡선이 전반적으로 분리되는 양상을 보였으며, T4군에서 생존확률이 상대적으로 더 빠르게 감소하는 경향이 관찰되었다. 이는 실제 임상 연구에서 보고되는 병기 증가에 따른 예후 악화 방향성과는 일치하나, 곡선 간 분리 정도는 제한적인 수준에 머물렀다. 조직학적 진단(Diagnosis)에 따른 생존곡선 비교에서는 여러 진단 범주 간 생존곡선이 상당 부분 중첩되어 나타났다. 일부 진단군에서 상대적으로 높은 또는 낮은 생존 경향이 관찰되기는 하였으나, 곡선 간 명확한 위계 구조나 지속적인 분리 양상은 뚜렷하지 않았다. 이는 조직학적 유형에 따른 예후 차이가 합성데이터 상에서는 충분히 구조화되어 반영되지 않았을 가능성을 시사한다. BRCA 상태에 따른 생존곡선 역시 병원성 변이(Pathogenic), 변이 불확실(VUS), 음성(Negative), 미상(Unknown) 집단 간에 전반적으로 유사한 형태를 보였으며, 장기 추적 구간에서도 뚜렷한 생존 차이는 관찰되지 않았다. 이는 실제 임상에서 보고되는 BRCA 상태와 예후 간의 복합적인 관계가 합성데이터에서는 단순화되어 표현되었을 가능성을 보여준다. 출산 경험 여부(Has_birth)에 따른 생존곡선 비교에서도 두 군의 생존곡선은 대부분의 추적 기간 동안 거의 중첩되어 나타났으며 출산 여부에 따른 생존 차이는 제한적인 수준에 그쳤다.



<그림 3.2> 주요 임상 변수들에 대한 KM곡선

<그림 3.3>은 주요 임상 변수에 따른 누적위험함수(cumulative hazard function)이다. 누적위험함수는 시간 경과에 따라 사망 위험이 누적되는 양상을 직접적으로 보여주며, Kaplan-Meier 생존곡선에서 관찰된 경향을 보완적으로 확인할 수 있다. 먼저 T 병기에 따른 비교에서는 T4군에서 누적위험이 전반적으로 더 빠르게 증가하는 경향이 관찰되었으며, 이는 Kaplan-Meier 생존곡선에서 나타난 T4군의 상대적으로 불리한 생존 양상과 일관된 결과이다. 조직학적 진단(Diagnosis)에 따른 누적위험함수 비교에서는 여러 진단군의 곡선이 상당 부분 중첩되어 나타났다. 일부 진단군에서 특정 시점 이후 누적위험의 증가 속도가 다소 차이를 보이기도 하였으나, 전반적으로 뚜렷한 분리 구조는 확인되지 않았다. 이는 조직학적 유형에 따른 위험 차이가 합성데이터 상에서는 명확한 누적위험 구조로 구현되지 않았음을 시사한다. BRCA 상태에 따른 누적위험함수 역시 병원성 변이, 변이 불확실, 음성 및 미상 집단 간에 전반적으로 유사한 증가 패턴을 보였으며, 장기 추적 구간에서도 집단 간 누적위험의 격차는 제한적이었다. 출산 경험 여부(Has_birth)에 따른 비교에서도 두 집단의 누적위험 곡선은 대부분의 추적 기간 동안 거의 겹치는 양상을 보였다.



<그림 3.3> 주요 임상 변수들에 대한 누적위험률 함수

3.2 Log-rank Test

로그-순위 검정(log-rank test)은 비모수적 검정으로, 두 개 이상의 집단 간 생존함수의 동일성 여부를 평가하기 위한 방법이다. 이 검정은 각 사건 발생 시점에서 관측된 사건 수와 기대 사건 수의 차이를 누적하여, 집단 간 생존 경험이 통계적으로 유의하게 다른지를 판단한다.

$$H_0 : S_1(t) = S_2(t) \text{대} H_1 : S_1(t) \neq S_2(t)$$

주요 임상·병리학적 변수에 대해 로그-순위 검정을 수행한 결과는 <표 3.2>와 같다. 분석에 포함된 모든 변수에서 통계적으로 유의한 생존곡선의 차이는 관찰되지 않았으며, 조직학적 진단(Diagnosis)의 경우 카이제곱 통계량이 가장 크게 나타났으나(χ^2 = 8.46), 유의확률은 0.133으로 일반적인 유의수준 0.05에는 도달하지 못하였다. 출산 경험 여부(Has_birth), BRCA 상태(BRCA_status), 폐경 여부(Menopause_status), 치료 유형(treat), 체질량지수 범주(BMI_group), 연령군(AGE_group) 등 주요 임상 변수 역시 모두 유의하지 않은 결과를 보였다. 호르몬 수용체 관련 변수(ER_clean, PR_clean, AR_clean) 및 이들의 조합으로 구성된 Hormone_status 또한 생존곡선 간의 유의한 차이를 보이지 않았으며, 수술 유형(Surgery_type) 역시 통계적으로 의미 있는 생존 차이를 나타내지 않았다. 전반적으로 모든

변수에서 p-value가 0.1을 크게 상회하여, 집단 간 생존함수가 동일하다는 귀무가설을 기각할 수 없다. 이러한 결과는 Kaplan-Meier 생존곡선 및 누적위험함수에서 관찰된 집단 간 곡선의 중첩 양상과 일관되며, 개별 임상 변수에 따른 생존 차이가 명확하게 분리되지 않음을 시사한다. 즉, 합성데이터는 전체 생존 추세와 시간에 따른 위험 누적 구조는 일정 수준 재현하고 있으나 실제 임상 데이터에서 기대되는 변수별 예후 차이를 통계적으로 유의한 수준으로 반영하지는 못한 것으로 판단된다.

<표 3.2> Log-rank Test 결과

variable	n_levels	n	events	chi2	p_value
Diagnosis	6	10000	3476	8.456402	0.132812
Has_birth	2	10000	3476	1.541097	0.214455
BRCA_status	4	10000	3476	4.186513	0.242016
Menopause_status	2	10000	3476	0.994582	0.318625
treat	16	10000	3476	15.514033	0.415063
BMI_group	4	10000	3476	1.893987	0.594699
AGE_group	3	10000	3476	0.740985	0.690394
PR_clean	3	10000	3476	0.315200	0.854191
ER_clean	3	10000	3476	0.238494	0.887589
Surgery_type	4	10000	3476	0.390683	0.942160
AR_clean	3	10000	3476	0.071592	0.964837
Hormone_status	9	10000	3476	2.180675	0.974966

3.3 Cox 비례위험모형

본 연구에서는 유방암 환자의 생존에 영향을 미치는 예후 인자를 규명하기 위하여 Cox 비례위험모형을 적용하였다. Cox 비례위험모형은 시간 t에서의 위험함수를 다음과 같이 정의한다.

$$h(t|X) = h_{0(t)}\exp(\beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p)$$

여기서 $h(t|X)$ 는 공변량 벡터 $X = (X_1, X_2, \ldots, X_p)$가 주어졌을 때의 조건부 위험함수이며, $h_{0(t)}$는 모든 공변량이 0일 때의 기저위험함수(baseline hazard function)이다. Cox 비례위험모형은 기저위험함수에 대해 특정 분포를 가정하지 않는 준모수적(semi-parametric) 모형으로, 공변량의 효과는 회귀계수 β_j를 통해 추정된다.

회귀계수의 지수화 값 $\exp(\beta_j)$는 위험비(hazard ratio, HR)로 해석되며, 이는 다른 공변량이 동일할 때 해당 공변량이 1단위 증가함에 따라 사건 발생 위험이 변화하는 비율을 의미한다.

3.3.1 단변량 Cox 비례위험모형

각 공변량의 개별적인 효과를 평가하기 위하여 공변량이 하나만 포함된 단변량 Cox 비례위

험모형을 적합하였다. 분석에는 결측값이 제거된 완전 사례만을 사용하였으며, 범주형 변수는 기준 범주(reference category)를 설정한 더미 변수 형태로 모형에 포함하였다.

단변량 Cox 비례위험모형 분석 결과, 대부분의 변수에서 통계적으로 유의한 결과가 관찰되지 않았다. 이에 본 연구에서는 유의수준 5% 기준에서 유의하거나, 유의수준 20% 기준에서 경계선 유의성을 보인 변수들을 중심으로 결과를 요약하였다.

<표 3.3>는 모든 공변량을 대상으로 수행한 단변량 Cox 비례위험모형 분석 결과 중, 조직학적 진단, T 병기, BRCA 유전자 상태, BMI 및 치료 유형과 같이 상대적으로 낮은 유의확률을 보인 주요 변수들의 결과를 요약하여 제시한 것이다.

조직학적 진단은 유의수준 5% 기준에서 통계적으로 유의한 관련성을 보였으며, T 병기, BRCA 유전자 상태, BMI 및 치료 유형은 유의수준 20% 기준에서 경계선 유의성을 보여 후속 다변량 Cox 비례위험모형 분석을 위한 후보 변수로 선정하였다.

<표3.3> 단변량 Cox 비례위험모형 분석 결과

변수	수준	coef	HR	SE	Z	p-값	신뢰구간 하한	신뢰구간 상한
조직학적 진단 (Diagnosis)	Metaplastic carcinoma (ref)	0.0000	1.0000	-	-	-	-	-
조직학적 진단 (Diagnosis)	Other	-0.0451	0.9559	0.1074	-0.4194	0.6749	0.7744	1.1800
조직학적 진단 (Diagnosis)	adenocarcinoma	-0.0999	0.9050	0.1070	-0.9331	0.3508	0.7337	1.1162
조직학적 진단 (Diagnosis)	inductal carcinoma	-0.1113	0.8947	0.0924	-1.2046	0.2284	0.7466	1.0723
조직학적 진단 (Diagnosis)	infiltrating duct carcinoma	-0.1567	0.8549	0.0915	-1.7131	0.0867	0.7146	1.0228
조직학적 진단 (Diagnosis)	lobular carcinoma	-0.2618	0.7696	0.1202	-2.1780	**0.0294**	0.6081	0.9741
T 병기 (T-stage)	T1 (ref)	0.0000	1.0000	-	-	-	-	-
T 병기 (T-stage)	T2	-0.0399	0.9608	0.0521	-0.7669	0.4432	0.8676	1.0641
T 병기 (T-stage)	T3	-0.0342	0.9664	0.1133	-0.3018	0.7628	0.7739	1.2068
T 병기 (T-stage)	T4	0.1920	1.2116	0.1361	1.4099	0.1586	0.9279	1.5821
BRCA 유전자 상태 (BRCA status)	Negative (ref)	0.0000	1.0000	-	-	-	-	-
BRCA 유전자 상태 (BRCA status)	Pathogenic	0.0627	1.0647	0.0459	1.3645	0.1724	0.9730	1.1649
BRCA 유전자 상태 (BRCA status)	Unknown	-0.0095	0.9905	0.0496	-0.1920	0.8477	0.8988	1.0916
BRCA	VUS	-0.0177	0.9824	0.0569	-0.3112	0.7556	0.8788	1.0983

유전자 상태 (BRCA status)								
BMI (연속형)	-	0.0045	1.0045	0.0033	1.3739	0.1695	0.9981	1.0109

3.3.2 공변량 포함하지 않은 경우

다변량 Cox 비례위험모형의 성능을 비교하기 위한 기준선으로서, 본 연구에서는 공변량이 포함되지 않은 Cox 비례위험모형(null model)을 적합하였다. 공변량이 없는 Cox 모형은 기저위험함수만을 포함하며, 다음과 같이 표현된다.

$$h(t) = h_{0(t)}$$

여기서 $h(t)$는 시간 t에서의 위험함수이며, $h_{0(t)}$는 모든 공변량이 고려되지 않은 상태에서의 기저위험함수(baseline hazard function)를 의미한다. 본 모형은 개별 공변량의 효과를 반영하지 않으므로, 이후 적합되는 다변량 Cox 비례위험모형의 예측 성능과 적합도를 비교하기 위한 기준 모형으로 활용된다.

Null Cox 비례위험모형은 총 10,000명의 환자를 대상으로 적합되었으며, 이 중 3,476건의 사건(사망)이 관찰되었다. 해당 모형의 C-index는 0.500으로 나타나, 생존 예측 성능이 무작위 추측(random prediction) 수준임을 확인하였다. 또한 부분우도 기반 AIC 값은 57,206.30으로 산출되었다. 이 값은 이후 모든 공변량을 포함한 모형 및 최종 선택 모형과의 상대적 비교를 위한 기준으로 사용하였다.

3.3.3 모든 공변량을 포함한 경우

단변량 Cox 비례위험모형 분석 결과와 임상적 중요성을 종합적으로 고려하여, 모든 후보 공변량을 동시에 포함한 다변량 Cox 비례위험모형(full model)을 적합하였다. 본 모형에는 병기(Stage), 연령(AGE), BMI, 생식 관련 변수(출산 여부, 폐경 여부), 조직학적 진단(Diagnosis), 호르몬 수용체 상태(ER, PR, AR), BRCA 유전자 상태, 흡연 여부(Smoke), 치료 유형(treat)이 공변량으로 포함되었다. 범주형 변수는 기준 범주(reference category)를 설정한 더미 변수 형태로 모형에 포함하였다.

Full Cox 비례위험모형은 총 10,000명의 환자를 대상으로 적합되었으며, 이 중 3,476건의 사건(사망)이 관찰되었다. 모형 적합 결과, C-index는 0.528로 나타나 공변량이 없는 Cox 모형(null model)의 C-index 0.500에 비해 예측 성능이 다소 향상되었음을 확인하였다. 반면 부분우도 기반 AIC 값은 57,248.88로 산출되었다.

Full model의 주요 추정 결과는 <표 3.4>에 제시하였다. 조직학적 진단 중 lobular carcinoma는 기준 범주 대비 사망 위험이 유의하게 낮은 것으로 나타났으며(HR = 0.76, p = 0.03), 이는 단변량 분석 결과와 일관된 경향을 보였다. 또한 infiltrating duct carcinoma는 유의수준 20% 기준에서 경계선 유의성을 보였다(p = 0.08). 반면 병기(Stage), BRCA 유전자 상태, BMI를 포함한 대부분의 공변량은 다변량 환경에서는 통계적으로 유의한 영향을 보이지 않았다.

이와 같이 Full model에서는 단변량 분석에서 관찰되었던 일부 변수의 효과가 다변량 분석에서 약화되거나 소실되는 경향이 관찰되었다. 이는 공변량 간 상관성 및 정보 중복의 영향으로 해석될 수 있으며, 특히 병기 관련 변수와 조직학적o임상적 변수 간의 연관성에 기인한 결

과로 판단된다. 이에 따라 본 연구에서는 모형의 복잡도를 줄이고 해석 가능성을 향상시키기 위하여 후진 제거법을 적용한 최종 Cox 비례위험모형을 추가로 도출하였다.

<표 3.4> Full Cox 다변량 모형 결과표

변수	수준	HR	95% 신뢰구간	p-값
조직학적 진단 (Diagnosis)	Metaplastic carcinoma (ref)	1.00	-	-
조직학적 진단 (Diagnosis)	lobular carcinoma	0.76	0.60 - 0.97	**0.03**
조직학적 진단 (Diagnosis)	infiltrating duct carcinoma	0.85	0.71 - 1.02	0.08
T 병기 (Stage)	T1 (ref)	1.00	-	-
T 병기 (Stage)	T4	1.21	0.93 - 1.58	0.16
BRCA 유전자 상태	Negative (ref)	1.00	-	-
BRCA 유전자 상태	Pathogenic	1.07	0.98 - 1.17	0.14
BMI (연속형)	-	1.00	1.00 - 1.01	0.19

3.3.4 후진 제거법을 이용한 변수 선택

모든 공변량을 포함한 다변량 Cox 비례위험모형은 변수 수가 많아 모형의 복잡도가 높고, 다수의 공변량이 통계적으로 유의하지 않은 것으로 나타났다. 이에 본 연구에서는 모형의 간명성과 해석 가능성을 향상시키기 위하여 부분우도 기반 AIC(Akaike Information Criterion)를 기준으로 한 후진 제거법(backward elimination)을 적용하였다.

후진 제거법은 초기 풀모형에서 하나의 공변량을 제거한 여러 후보 모형을 반복적으로 적합한 뒤, AIC 값이 가장 감소하는 변수를 제거하는 방식으로 진행하였다. 더 이상 AIC가 감소하지 않을 경우 변수 제거를 중단하였다. 분석에는 TNM 병기(T, N, M), 연령(AGE), BMI, 생식 관련 변수(출산 여부, 폐경 여부), 조직학적 진단(Diagnosis), 호르몬 수용체 상태(ER, PR, AR), BRCA 유전자 상태, 흡연 여부(Smoke), 치료 유형(treat) 등 총 14개의 공변량이 초기 모형에 포함되었다.

후진 제거 과정은 총 13단계에 걸쳐 진행되었으며, 제거 순서는 <표 3.5>에 제시하였다. 분석 결과, 치료 유형, 호르몬 수용체 변수, 흡연 여부, TNM 병기 변수, 연령, 조직학적 진단, BRCA 유전자 상태, 생식 관련 변수 등이 순차적으로 제거되었고, 최종적으로 BMI만이 AIC 기준에서 가장 적합한 공변량으로 선택되었다.

<표 3.5> 후진제거법 결과 요약

단계	제거된 변수	남은 변수 수	AIC_partial	C-index
0	시작 모형	14	57249.79	0.530
1	treat	13	57235.97	0.524
2	AR_clean	12	57232.02	0.524
3	ER_clean	11	57228.24	0.523
4	PR_clean	10	57224.56	0.523
5	Smoke	9	57221.26	0.523
6	T_stage	8	57218.01	0.521
7	N_stage	7	57214.69	0.520
8	M_stage	6	57212.76	0.520
9	AGE	5	57211.10	0.519
10	Diagnosis	4	57209.55	0.516
11	BRCA_status	3	57208.02	0.510
12	Menopause_status	2	57206.93	0.509
13	Has_birth	1	**57206.41**	0.507

후진 제거가 완료된 최종 모형에는 BMI 단일 공변량만 포함되었으며, 이 모형의 C-index는 0.507, AIC_partial 값은 57,206.41로 나타났다. 이는 공변량이 없는 Cox 모형(null model)의 AIC 값(57,206.30)과 거의 유사한 수준으로, 다수의 임상o병리학적 변수를 포함하더라도 생존 예측 성능이 크게 향상되지 않음을 시사한다.

이러한 결과는 본 연구에서 사용한 GAN 기반 합성 데이터가 개별 변수의 분포는 비교적 잘 재현하였으나, 생존 시간과 사망 여부에 영향을 미치는 복합적인 인과 구조를 충분히 보존하지 못했을 가능성을 시사한다. 따라서 후진 제거법 결과는 통계적으로는 합리적이나, 임상적 해석에는 주의가 필요하다.

3.4 최종모형

후진 제거법을 적용한 결과, AIC 기준에서 BMI 단일 공변량을 포함한 모형이 가장 간명한 모형으로 선택되었다. 그러나 본 연구에서는 통계적 기준뿐만 아니라 임상적 중요성과 단계별 생존분석 결과(Kaplan-Meier 분석 및 단변량 Cox 비례위험모형 분석)를 종합적으로 고려하여 최종 Cox 비례위험모형을 구성하였다. 이에 따라 단변량 분석에서 경계선 유의성 또는 유의성을 보였으며, 임상적으로 중요한 변수로 판단되는 BMI, 조직학적 진단(Diagnosis), BRCA 유전자 상태를 공변량으로 포함한 모형을 최종 모형으로 선정하였다.

최종 Cox 비례위험모형은 총 10,000명의 환자를 대상으로 적합되었으며, 이 중 3,476건의 사망 사건이 관찰되었다. 범주형 변수는 기준 범주(reference category)를 설정한 더미 변수 형태로 모형에 포함하였고, BMI는 연속형 변수로 투입하였다. 최종 모형의 C-index는 0.518로 나타나 공변량이 없는 Cox 모형(null model)의 C-index 0.500에 비해 예측 성능이 다소 향상되었으며, 부분우도 기반 AIC 값은 57,209.70으로 산출되었다.

최종 Cox 비례위험모형의 추정 결과는 <표 3.6>에 제시하였다. 조직학적 진단 중 lobular carcinoma는 기준 범주 대비 사망 위험이 유의하게 낮은 것으로 나타났으며(HR = 0.77, p = 0.03), 이는 단변량 분석 및 다변량 분석 전반에서 일관된 결과를 보였다. 또한 infiltrating duct carcinoma는 유의수준 20% 기준에서 경계선 유의성을 보였다(p = 0.08). BMI는 증가에 따라 사망 위험이 증가하는 경향을 보였으나 통계적으로 유의하지는 않았다(p = 0.16). BRCA 유전자 상태는 Pathogenic 범주에서 위험비가 증가하는 경향을 보였으나 유의수준 5%에서는 통계적으로 유의하지 않았다.

이와 같이 최종 모형은 통계적 유의성만을 기준으로 한 변수 선택이 아니라, 단계적 생존분석 결과와 임상적 해석 가능성을 종합적으로 반영하여 구성되었으며, 이는 GAN 기반 합성데이터를 활용한 생존분석의 특성과 한계를 동시에 반영한 결과로 해석할 수 있다.

[2]<표 3.6> 최종 선택변수 기반 Cox비례위험모형

변수	수준	회귀계수 (coef)	위험비 (HR)	표준오차 (SE)	z값	p-값	95% 신뢰구간 하한	95% 신뢰구간 상한
BRCA 유전자 상태	Negative (ref)	0.000	1.000	-	-	-	-	-
BRCA 유전자 상태	Pathogenic	0.070	1.07	0.05	1.44	0.15	0.98	1.17
BRCA 유전자 상태	Unknown	-0.010	0.99	0.05	-0.14	0.89	0.90	1.09
BRCA 유전자 상태	VUS	-0.020	0.98	0.06	-0.27	0.79	0.88	1.10

2) HR은 위험비(hazard ratio)를 의미하며, 범주형 변수의 HR은 기준 범주 대비 위험비이다. 신뢰구간은 95% 신뢰구간이다.

조직학적 진단 (Diagnosis)	Metaplastic carcinoma (ref)	0.000	1.000	-	-	-	-	-
조직학적 진단 (Diagnosis)	Other	-0.040	0.96	0.11	-0.41	0.68	0.77	1.18
조직학적 진단 (Diagnosis)	adenocarcinoma	-0.100	0.90	0.11	-0.95	0.34	0.73	1.11
조직학적 진단 (Diagnosis)	inductal carcinoma	-0.120	0.89	0.09	-1.25	0.21	0.74	1.07
조직학적 진단 (Diagnosis)	infiltrating duct carcinoma	-0.160	0.85	0.09	-1.75	0.08	0.71	1.02
조직학적 진단 (Diagnosis)	lobular carcinoma	-0.260	0.77	0.12	-2.19	0.03	0.61	0.97
BMI (연속형)	-	0.004	1.00	0.00	1.39	0.16	1.00	1.01

최종 Cox 비례위험모형은 다음과 같이 표현된다.

$$h(t|X) = h_{0(t)}\exp(\beta_{BMI} \cdot BMI + \beta_{BRCA(Pathogenic)}) \cdot I(BRCA = Pathogenic) + \beta_{BRCA(Unknown)} \cdot I(BRCA = Unknown) + \beta_{BRCA(VUS)} \cdot I(BRCA = VUS) + \Sigma_k \beta_{Diagnosis}, k \cdot I(Diagnosis = k))$$

여기서 $h_{0(t)}$는 기저위험함수(baseline hazard function)를 의미하며, $I(\cdot)$는 지시함수(indicator function)이다. 조직학적 진단 변수의 기준 범주는 Metaplastic carcinoma이고, BRCA 유전자 상태의 기준 범주는 Negative이다.

3.4.1 최종모형 변수의 상호작용 효과 검정

최종 Cox 비례위험모형에서 공변량 간 상호작용(interaction) 존재 여부를 확인하기 위하여, 주효과(main effects)만 포함한 기본 모형과 상호작용항을 추가한 확장 모형을 각각 적합한 뒤 우도비 검정(likelihood ratio test, LRT)으로 비교하였다. 분석에는 생존시간과 사건변수, BRCA 유전자 상태, 조직학적 진단, BMI가 모두 관측된 완전 사례만 사용하였으며, 표본수는 10,000명(사건수 3,476건)이었다. 기본 모형은 다음과 같이 주효과만을 포함하였다.

$$h(t|X) = h_{0(t)}\exp(\beta_{BMI} \cdot BMI + \Sigma_j \beta_{BRCA,j} \cdot I(BRCA = j) + \Sigma_k \beta_{Dx,k} \cdot I(Diagnosis = k))$$

이 기본 모형의 적합 결과, 로그우도는 -28595.85, 부분우도 기반 AIC는 57,209.70, C-index는 0.518로 나타났다.

상호작용 검정은 한 번에 여러 상호작용을 동시에 추가하지 않고, (1) BRCA×Diagnosis, (2) BMI×Diagnosis, (3) BMI×BRCA의 세 가지 후보 상호작용을 각각 기본 모형에 하나씩 추가하여 평가하였다. 각 확장 모형에 대해, 기본 모형 대비 우도비 통계량 $LR = 2(\ell_1 - \ell_0)$을 계

산하고, 자유도는 추가된 모수 개수(Δdf)로 설정하여 카이제곱 분포에 기반한 p-값을 산출하였다. 또한 상호작용항 추가에 따른 AIC 변화(ΔAIC) 및 C-index 변화(ΔC)도 함께 비교하였다.

분석 결과는 <표 3.7>과 같다. BRCA×Diagnosis 상호작용 모형은 LRT p-값이 0.086으로 유의수준 5%에서는 유의하지 않았으며, AIC 또한 7.11 증가하여(적합도 악화) 상호작용 추가의 이점이 제한적이었다. 다만 C-index는 0.0069 증가하였다. BMI×Diagnosis 및 BMI×BRCA 상호작용 모형은 각각 LRT p-값이 0.412, 0.219로 나타나 유의하지 않았고, AIC 역시 증가하였다. 따라서 본 연구에서는 상호작용항을 포함하지 않은 주효과 모형을 최종 모형으로 채택하였다.

[3]<표 3.7> 상호작용항 추가에 따른 우도비 검정 및 적합도 변화

상호작용 검정	추가항	Δdf	LR 통계량	LRT p-값	AIC(기본)	AIC(상호작용)	ΔAIC	C-index(기본)	C-index(상호작용)	ΔC	상호작용항 최소 p-값
BRCA × Diagnosis	C(BRCA_status):C(Diagnosis)	15	22.8906	0.0865	57209.6960	57216.8054	+7.1094	0.5183	0.5252	+0.0069	0.0346
BMI × BRCA	BMI:C(BRCA_status)	3	4.4233	0.2192	57209.6960	57211.2727	+1.5767	0.5183	0.5195	+0.0012	0.1893
BMI × Diagnosis	BMI:C(Diagnosis)	5	5.0321	0.4120	57209.6960	57214.6640	+4.9679	0.5183	0.5211		

3.4.2 비례위험가정 검정 및 잔차 진단

Cox 비례위험모형은 공변량의 효과가 시간에 따라 일정하다는 비례위험가정(proportional hazards assumption)을 전제로 한다. 즉, 두 개체 간 위험비(hazard ratio)는 시간 t에 의존하지 않고 상수로 유지된다고 가정한다. 본 절에서는 최종 Cox 비례위험모형이 이러한 가정을 만족하는지 확인하기 위하여 통계적 검정, 시각적 진단, 잔차 분석을 단계적으로 수행하였다.

최종 모형에 포함된 공변량은 BRCA 유전자 상태, 조직학적 진단(Diagnosis), BMI이며, 모든 분석은 해당 변수들이 결측되지 않은 완전 사례(n = 10,000, 사건수 = 3,476)를 대상으로 수행하였다.

3.4.2.1 Schoenfeld 잔차 기반 비례위험가정 검정

비례위험가정에 대한 1차적인 검정으로 Schoenfeld 잔차를 이용한 통계적 검정을 실시하였다. Schoenfeld 잔차는 각 사건 시점에서 공변량의 관측값과 모형이 예측한 기대값의 차이를 나타내며, 비례위험가정이 성립할 경우 잔차는 시간과 독립적이어야 한다.

본 연구에서는 check_assumptions 함수를 사용하여 각 공변량에 대해 Schoenfeld 잔차와 시간 간의 상관성을 검정하였다. 유의수준 5%를 기준으로 판단한 결과, BRCA 유전자 상태, 조직학적 진단, BMI 모두에서 비례위험가정이 유의하게 위배되지 않음을 확인하였다.

---

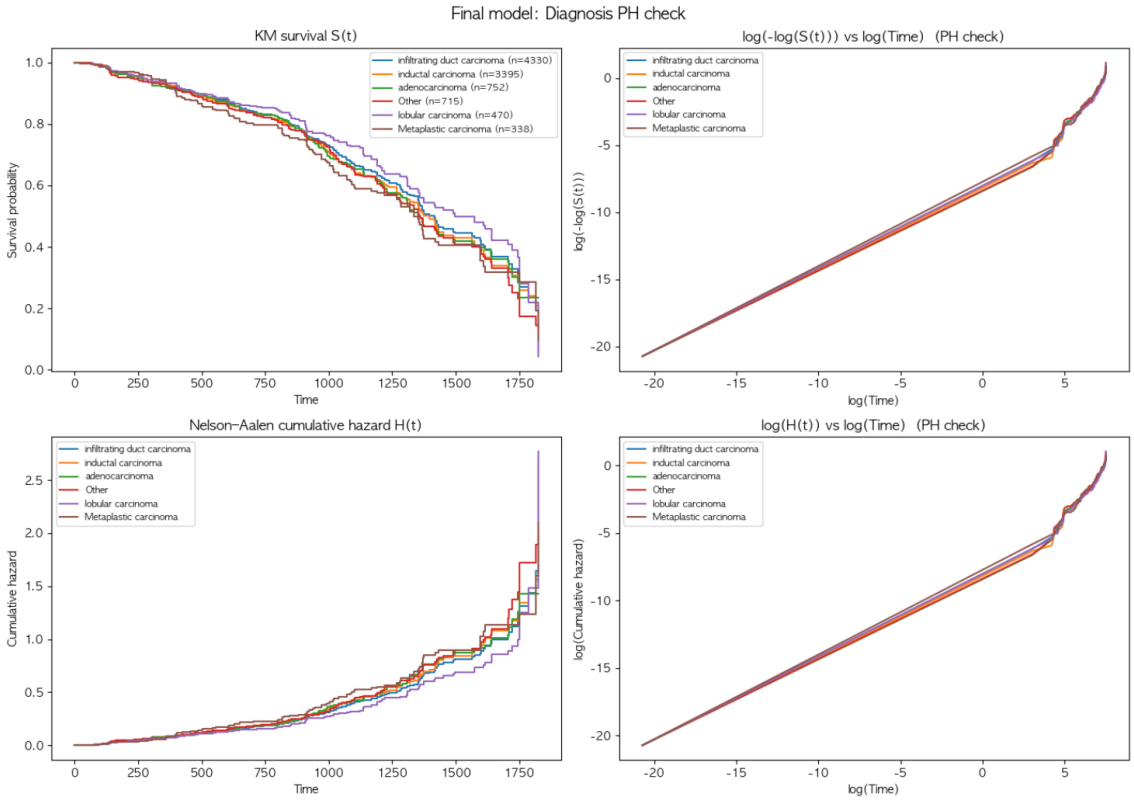
3) ΔAIC가 양수(+)이면 상호작용항 추가로 AIC가 증가하여 모형 적합도가 악화됨을 의미한다. LRT는 기본 모형과 상호작용항을 포함한 확장 모형의 중첩(nested) 관계를 이용한 우도비 검정이다.

따라서 통계적 검정 결과에 근거할 때, 최종 Cox 비례위험모형은 비례위험가정을 전반적으로 만족하는 것으로 판단하였다.

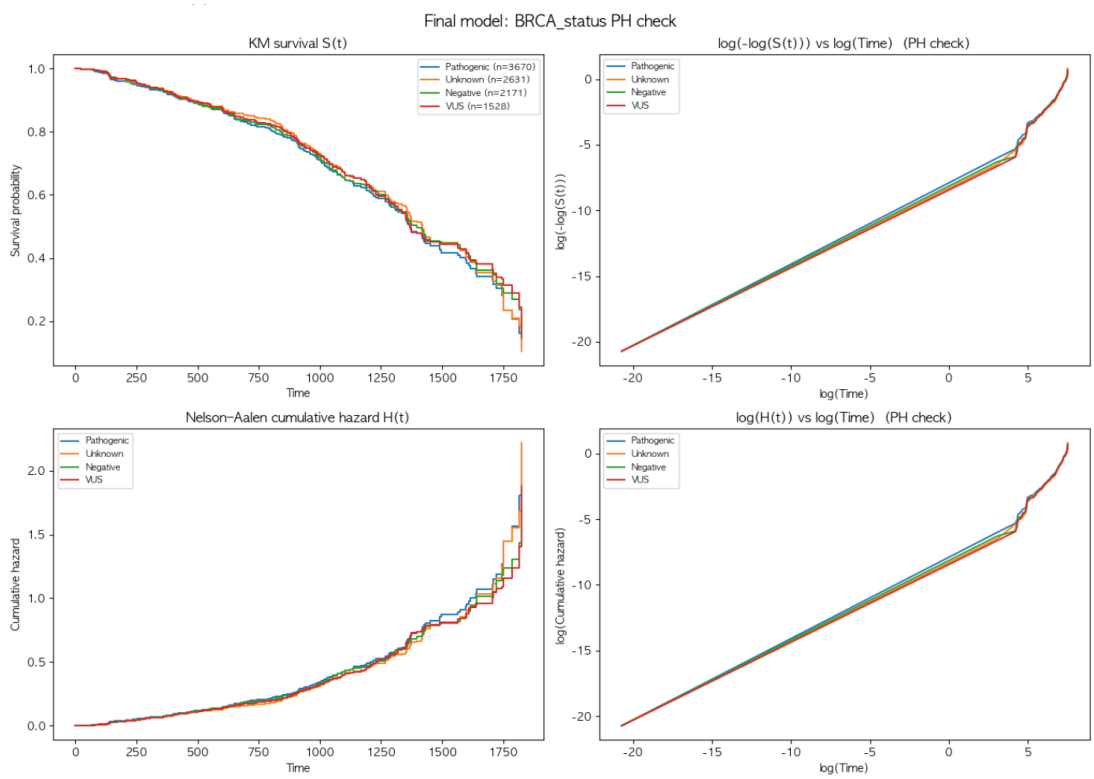
3.4.2.2 로그-로그 생존곡선을 이용한 시각적 진단

비례위험가정은 시각적으로도 점검할 수 있다. Kaplan-Meier 생존함수 S(t)에 대해 $\log-\log[S(t)] vs \log(t)$에 대해 도시하였을 때, 각 범주별 곡선이 대체로 평행한 형태를 보이면 비례위험가정이 성립한다고 해석한다.

본 연구에서는 범주형 공변량인 조직학적 진단(Diagnosis)과 BRCA 유전자 상태에 대해 Kaplan-Meier 생존곡선과 로그-로그 생존곡선을 함께 제시하였다. 그 결과, 각 범주별 $\log(-\log(S(t)))$곡선은 전반적으로 큰 교차 없이 유사한 기울기를 유지하였으며, 이는 시간에 따른 공변량 효과의 급격한 변화가 없음을 시사한다. 해당 결과는 <그림 3.4>와 <그림 3.5>에 제시하였다.



<그림 3.4> 조직학적 진단에 따른 Kaplan-Meier 생존곡선 및 로그-로그 생존곡선



<그림 3.5> BRCA 유전자 상태에 따른 Kaplan-Meier 생존곡선 및 로그-로그 생존곡선

3.4.2.3 누적위험함수 기반 진단

추가적으로 Nelson-Aalen 방법을 이용하여 누적위험함수 H(t)를 추정하고, $\log(H(t))$와 $\log(t)$의 관계를 시각화하였다. 비례위험가정이 성립할 경우, 각 범주별 $\log(H(t))$ 곡선 역시 시간에 대해 대체로 평행한 형태를 보인다.

분석 결과, 조직학적 진단 및 BRCA 유전자 상태에 따른 누적위험함수 그래프에서도 뚜렷한 교차나 비선형 패턴은 관찰되지 않았다. 이는 앞선 로그-로그 생존곡선 분석 결과와 일관되며, 비례위험가정이 대체로 만족됨을 시사한다.

3.4.2.4 Martingale 잔차 및 Deviance 잔차 분석

모형 적합의 추가적인 진단을 위하여 Martingale 잔차와 deviance 잔차를 계산하였다. Martingale 잔차는 다음과 같이 정의된다.

$$r_M = \delta_i - \hat{H}_{0(t_i)} \exp(x_i \hat{\beta})$$

여기서 δ_i는 사건 발생 여부, $\hat{H}_{0(t_i)}$는 개인 i의 관측 시점에서의 추정된 기저 누적위험함수이다. Deviance 잔차는 Martingale 잔차를 대칭적으로 변환한 값으로, 이상치(outlier)를 보다 명확히 식별하는 데 사용된다.

잔차의 기술통계량을 확인한 결과, Martingale 잔차와 deviance 잔차 모두 극단적인 분산이나 비정상적인 패턴을 보이지 않았으며, 특정 관측치가 모형 적합을 과도하게 지배하는 현상은 관찰되지 않았다. 이는 최종 모형이 자료 전반에 걸쳐 비교적 안정적으로 적합되었음을 시사한다. 해당 잔차의 요약 통계량은 <표 3.8>에 제시하였다.

<표 3.8> Martingale 잔차 및 Deviance 잔차 요약 통계량

잔차 종류	관측치 수	평균	표준편차	최소값	제1사분위수 (Q1)	중앙값 (Median)	제3사분위수 (Q3)	최대값
Martingale 잔차	10,000	0.000	0.586	-2.171	-0.350	-0.099	0.559	0.998
Deviance 잔차	10,000	-0.077	1.057	-2.084	-0.822	-0.431	0.720	3.275

Martingale 잔차의 평균은 0에 근접하며 분포가 극단적으로 치우치지 않았고, deviance 잔차 역시 제한된 범위 내에서 분포하였다. 이는 특정 관측치가 모형 적합에 과도한 영향을 미치지 않음을 시사하며, 최종 Cox 비례위험모형이 자료 전반에 대해 안정적으로 적합되었음을 보여준다.

Schoenfeld 잔차 기반 통계적 검정, 로그-로그 생존곡선 및 누적위험함수에 대한 시각적 진단, 그리고 Martingale 및 deviance 잔차 분석을 종합한 결과, 최종 Cox 비례위험모형은 비례위험가정을 전반적으로 만족하는 것으로 판단되었다. 또한 잔차 분석에서도 모형 적합에 중대한 문제는 발견되지 않았다.

이에 따라 본 연구에서는 해당 모형을 유효한 Cox 비례위험모형으로 채택하였으며, 이후 장에서는 본 연구 결과의 해석과 GAN 기반 합성데이터를 활용한 생존분석의 한계에 대해 논의한다.

4. 결론 및 한계

4.1 분석 결과 요약

본 연구에서는 합성 데이터 기반 유방암 생존 자료를 대상으로 Cox 비례위험모형을 적용하여 예후 인자를 단계적으로 탐색하였다. Kaplan-Meier 곡선과 로그-순위 검정을 통해 기초적인 생존 차이를 확인한 후, 단변량 Cox 모형, 공변량을 모두 포함한 풀모형, 그리고 AIC 기준 후진 제거법을 적용한 축소모형으로 분석을 확장하였다.

단변량 Cox 비례위험모형에서는 대부분의 임상적ㆍ병리학적 변수가 통계적으로 유의하지 않았으며, 조직학적 진단 중 lobular carcinoma만이 기준 범주 대비 사망 위험 감소와 유의한 관련성을 보였다($p<0.05$). BMI와 BRCA 병원성 변이는 유의수준 20% 기준에서 경계선 수준의 효과를 보였다.

임상적으로 중요하다고 알려진 변수들을 모두 포함한 풀모형의 C-index는 약 0.53으로 나타났으나, 다수의 공변량이 유의하지 않았고 전반적인 예측 성능 향상은 제한적이었다. 이는 합성 데이터 환경에서 공변량과 생존시간 간의 연결 구조가 약함을 시사한다.

4.2 변수 선택 결과 및 최종모형

풀모형의 복잡도와 제한적인 설명력을 고려하여, AIC(partial likelihood 기반)를 기준으로 한 후진 제거법을 적용하였다. 그 결과 대부분의 공변량이 단계적으로 제거되었으며, BMI 단일 변수만이 통계적 기준(AIC)에 의해 최종 선택되었다.

그러나 본 연구에서는 통계적 기준만으로 최종모형을 결정하기보다는, 단계별 분석 결과와 임상적 중요도를 함께 고려하였다. 이에 따라 단변량 분석에서 상대적으로 일관된 효과를 보였던 조직학적 진단(Diagnosis)과 BRCA 유전자 상태(BRCA_status), 그리고 후진 제거 과정에서 가장 마지막까지 유지된 BMI를 포함한 축소모형을 최종모형으로 설정하였으며, 최종 Cox 비례위험모형은 다음과 같이 표현된다.

$$h(t|X) = h_{0(t)}\exp(\beta_1 \cdot BMI + \Sigma_k \beta_{2k} \cdot I(Diagnosis = k) + \Sigma_j \beta_{3j} \cdot I(BRCAstatus = j))$$

최종모형의 C-index는 약 0.52로 나타났으며, 이는 풀모형 대비 큰 성능 저하 없이 공변량 수를 크게 줄인 결과이다. 조직학적 진단 중 lobular carcinoma는 기준 조직형 대비 사망 위험이 유의하게 낮았으며($p<0.05$), BMI와 BRCA 병원성 변이는 통계적으로는 경계선 수준의 효과를 보였다.

4.3 상호작용 분석 결과

최종모형에 포함된 공변량 간의 효과 수정 가능성을 평가하기 위해 상호작용 분석을 수행하였다. 기본 주효과 모형에 상호작용 항을 하나씩 추가하고 로그우도비 검정을 통해 비교한 결과, BRCA 유전자 상태와 조직학적 진단 간 상호작용에서 유의수준 10% 기준의 경계선 유의성이 관찰되었다.

그러나 해당 상호작용 모형은 AIC가 증가하고 자유도 대비 설명력 개선이 제한적이었으며, 다른 상호작용 항들 역시 통계적o예측적 이점이 확인되지 않았다. 이에 따라 본 연구에서는 상호작용 항을 포함하지 않은 주효과 모형을 최종모형으로 채택하였다.

4.4 모형 적합성 및 비례위험 가정 검정

최종모형의 적합성을 평가하기 위해 비례위험 가정 검정과 잔차 분석을 수행하였다. Schoenfeld 잔차 기반 검정 결과, 모든 공변량에서 비례위험 가정 위반의 통계적 증거는 관찰되지 않았다.

Martingale 잔차와 Deviance 잔차 분석에서도 평균이 0에 가까운 분포를 보였으며, 뚜렷한 이상치나 체계적인 패턴은 확인되지 않았다. 또한 범주형 변수에 대해 Kaplan-Meier 생존곡선, Nelson-Aalen 누적위험곡선, log(−log(S(t))) 대 log(time) 플롯을 함께 검토한 결과, 전반적으로 곡선의 평행성이 유지되어 비례위험 가정이 시각적으로도 타당함을 확인하였다.

4.5 한계점

본 연구의 결과는 다음과 같은 한계를 가진다. 첫째, 실제 임상 데이터가 아닌 합성 데이터를 사용하였다는 점이다. 이로 인해 병기, 치료 변수 등 임상적으로 중요한 요인들이 통계적으로 유의하지 않게 나타났으며, 이는 임상적 효과의 부재라기보다 데이터 생성 구조의 한계로 해석하는 것이 타당하다. 둘째, AIC 기준 변수 선택 결과가 실제 임상 연구에서 기대되는 결과와 차이를 보였으며, 이에 따라 통계적 기준과 임상적 판단을 병행할 필요성이 확인되었다.

4.6 최종 결론

본 연구는 합성 데이터 환경에서 KM, log-rank test, Cox 비례위험모형을 적용하여 예후인자 탐색, 변수 선택, 상호작용 분석, 그리고 모형 진단에 이르는 생존분석의 전 과정을 체계적으로 제시하였다. 분석 결과 자체보다는, 합성 데이터 기반 생존분석에서 통계적 유의성 해석과 모형 선택에 요구되는 주의점을 실증적으로 보여주었다는 점에 의의가 있다.

본 연구에서 제시한 분석 프레임워크는 실제 임상 데이터에 적용 가능하며, 향후 연구에서는 실제 환자 자료 또는 보다 정교한 합성 데이터 생성 기법을 활용한 추가 검증이 필요할 것이다.

참고문헌

김재희. (2025.). R을 이용한 생존분석 기초. 자유아카데미.

공경엽. (2006). 유방암에 있어서 예후 및 치료 예측인자. Journal of the Korean Society for Breast Screening, 3(1), 5-12.

윤성욱, 우혜경, 김정은. (2021-06-23). 유방암 생존 예측: 특성 선택 및 예측 모델 비교, 그리고 유전적 특성의 효과. 한국정보과학회 학술발표논문집, 제주.

"BMI 25㎏/㎡ 이상 비만 여성, 유방암 중증도 높다" 2021.04.20. 게시
http://www.doctorsnews.co.kr/news/articleView.html?idxno=139116

부록

```
import pandas as pd
import numpy as np
import seaborn as sns
from lifelines import KaplanMeierFitter, CoxPHFitter, WeibullAFTFitter
from lifelines.statistics import logrank_test, multivariate_logrank_test
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv("Adjusted_synbreast_trainset.csv")
exclude_cols = ['ER', 'PR', 'AR']
for col in df.columns:
    if col not in exclude_cols:
        df[col] = df[col].replace([99, 999, 9], np.nan)
df['Age of Menarch'] = df['Age of Menarch'].fillna(df['Age of Menarch'].median())

df.loc[(df['Age of Menopause'] < 30) | (df['Age of Menopause'] > 60),
        'Age of Menopause'] = np.nan
df['Menopause_status'] = df['Age of Menopause'].notna().astype(int)
menopause_median = df.loc[df['Menopause_status']==1, 'Age of
Menopause'].median()
df.loc[df['Menopause_status']==1, 'Age of Menopause'] = \
    df.loc[df['Menopause_status']==1, 'Age of
Menopause'].fillna(menopause_median)

df['Has_birth'] = (df['Number of births'] > 0).astype(int)
birth_median = df.loc[df['Has_birth']==1, 'Age at first birth'].median()
df.loc[df['Has_birth']==1, 'Age at first birth'] = \
    df.loc[df['Has_birth']==1, 'Age at first birth'].fillna(birth_median)
```

```
df.loc[df['Has_birth']==0, 'Age at first birth'] = np.nan

for var in ['ER', 'PR', 'AR']:
    if var in df.columns:
        if var == 'AR':
            df[f'{var}_clean'] = df[var].replace(9, np.nan)
            mode_val = df[f'{var}_clean'].mode()[0]
            df[f'{var}_clean'] = df[f'{var}_clean'].fillna(mode_val)
        else:
            df[f'{var}_clean'] = df[var].copy()

        df[f'{var}_clean'] = df[f'{var}_clean'].astype('category')

if 'ER_clean' in df.columns and 'PR_clean' in df.columns:
    df['Hormone_status'] = (
        df['ER_clean'].astype(str) + '_' + df['PR_clean'].astype(str)
    ).astype('category')

histology_cols = ['inductal carcinoma', 'infiltrating duct carcinoma',
                  'adenocarcinoma', 'lobular carcinoma', 'Metaplastic
carcinoma']

def get_primary_diagnosis(row):
    for col in histology_cols:
        if row[col] == 1:
            return col
    return 'Other'

df['Diagnosis'] = df[histology_cols].apply(get_primary_diagnosis,
axis=1).astype('category')

import re

t_cols = [c for c in df.columns if re.fullmatch(r"T[1-4].*", c)]
n_cols = [c for c in df.columns if re.fullmatch(r"N[1-3].*", c)]
m_cols = [c for c in df.columns if c.startswith("M1")]

def get_T_stage(row):
    if any(row[col] == 1 for col in t_cols if 'T4' in col): return 'T4'
    elif any(row[col] == 1 for col in t_cols if 'T3' in col): return 'T3'
    elif any(row[col] == 1 for col in t_cols if 'T2' in col): return 'T2'
    elif any(row[col] == 1 for col in t_cols if 'T1' in col): return 'T1'
    else: return 'T0'
```

```
def get_N_stage(row):
    if any(row[col] == 1 for col in n_cols if 'N3' in col): return 'N3'
    elif any(row[col] == 1 for col in n_cols if 'N2' in col): return 'N2'
    elif any(row[col] == 1 for col in n_cols if 'N1' in col): return 'N1'
    else: return 'N0'

df['T_stage'] = df[t_cols].apply(get_T_stage, axis=1).astype('category')
df['N_stage'] = df[n_cols].apply(get_N_stage, axis=1).astype('category')

# M stage (M0 / M1)
if len(m_cols) > 0:
    df['M_stage'] = (
        df[m_cols].max(axis=1)
        .map({1: 'M1', 0: 'M0'})
        .astype('category')
    )
else:
    df['M_stage'] = pd.Series('M0', index=df.index, dtype='category')

def get_stage_simple(row):
    T, N, M = row['T_stage'], row['N_stage'], row['M_stage']
    if M == 'M1': return 4
    if T == 'T4' or N == 'N3' or (T == 'T3' and N in ['N1', 'N2']): return 3
    if T in ['T2', 'T3'] or N == 'N1': return 2
    if T == 'T1' and N == 'N0': return 1
    return 0

df['Stage'] = df.apply(get_stage_simple, axis=1).astype('category')

def get_stage_simple(row):
    T, N, M = row['T_stage'], row['N_stage'], row['M_stage']
    if M == 'M1': return 4
    if T == 'T4' or N == 'N3' or (T == 'T3' and N in ['N1', 'N2']): return 3
    if T in ['T2', 'T3'] or N == 'N1': return 2
    if T == 'T1' and N == 'N0': return 1
    return 0

df['Stage'] = df.apply(get_stage_simple, axis=1).astype('category')

brca_cols = ['BRCA1 PVs', 'BRCA1 VUS', 'BRCA2 PVs', 'BRCA2 VUS']

def get_brca_status(row):
    if pd.isna(row['BRCA1 PVs']) and pd.isna(row['BRCA2 PVs']):
        return 'Unknown'
```

```
    elif row['BRCA1 PVs'] == 1 or row['BRCA2 PVs'] == 1:
        return 'Pathogenic'
    elif row['BRCA1 VUS'] == 1 or row['BRCA2 VUS'] == 1:
        return 'VUS'
    else:
        return 'Negative'

df['BRCA_status'] = df[brca_cols].apply(get_brca_status, axis=1).astype('category')

df['BMI'] = df['Weight'] / (df['Height'] / 100) ** 2

def bmi_category(x):
    if x < 18.5: return 'Underweight'
    elif x < 23: return 'Normal'
    elif x < 25: return 'Overweight'
    else: return 'Obese'

df['BMI_group'] = df['BMI'].apply(bmi_category).astype('category')

df['AGE_group'] = pd.cut(df['AGE'], bins=[0, 50, 65, 100],
                         labels=['Young', 'Middle', 'Old']).astype('category')

df['Surgery_type'] = 0
df.loc[df['BCS']==1, 'Surgery_type'] = 1
df.loc[df['Mestectomy']==1, 'Surgery_type'] = 1
df.loc[df['pan hysterosalpingo oophorectomy']==1, 'Surgery_type'] += 2

def get_treatment(row):
    surg = row['BCS']==1 or row['Mestectomy']==1
    chemo = row['Chemotherapy']==1
    radio = row['Radiation Therapy']==1
    hormone = row['Hormone therapy']==1
    combo = []
    if surg: combo.append('Surgery')
    if chemo: combo.append('Chemo')
    if radio: combo.append('Radio')
    if hormone: combo.append('Hormone')
    return '+'.join(combo) if combo else 'None'

df['treat'] = df.apply(get_treatment, axis=1).astype('category')

df['Alcohol'] = (df['Type of Drink'] != 99).astype(int)

drop_cols = (
```

```
    ['No'] + histology_cols + t_cols + n_cols + m_cols + brca_cols +
    ['ER', 'PR', 'AR'] +
    ['BCS', 'Mestectomy', 'pan hysterosalpingo oophorectomy'] +
    ['Height', 'Weight', 'Marriage', 'COCP', 'HRT', 'Type of Drink']
)

drop_cols = [col for col in drop_cols if col in df.columns]
df_final = df.drop(columns=drop_cols)
df_final = df_final.drop(columns=['Alcohol'], errors='ignore')
cat_cols = [
    'ER_clean', 'PR_clean', 'AR_clean',
    'Hormone_status',
    'Diagnosis',
    'T_stage', 'N_stage', 'M_stage',
    'Stage',
    'BRCA_status',
    'BMI_group',
    'AGE_group',
    'treat',
    'Has_birth',
    'Menopause_status',
    'Surgery_type'
]

cat_cols = [c for c in cat_cols if c in df_final.columns]
df_final[cat_cols] = df_final[cat_cols].astype('category')

outcome_cols = ['Survival period', 'Death']

num_cols = [
    c for c in df_final.columns
    if c not in cat_cols + outcome_cols
]

df_final[num_cols] = df_final[num_cols].apply(
    pd.to_numeric, errors='coerce'
)
from lifelines import KaplanMeierFitter
from lifelines.statistics import multivariate_logrank_test

time_col = "Survival period"
event_col = "Death"

kmf = KaplanMeierFitter()
```

```
def km_summary_by_group(df, group_col, title):
    print(f"\n=== KM by {title} ({group_col}) ===")

    tmp = df[[time_col, event_col, group_col]].dropna()

    if str(tmp[group_col].dtype) == "category":
        levels = list(tmp[group_col].cat.categories)
    else:
        levels = sorted(tmp[group_col].unique())

    for lv in levels:
        mask = (tmp[group_col] == lv)
        n = int(mask.sum())
        if n == 0:
            continue

        kmf.fit(
            durations=tmp.loc[mask, time_col],
            event_observed=tmp.loc[mask, event_col],
            label=f"{title} {lv}"
        )
        print(f"{title} {lv}: n={n}, median={kmf.median_survival_time_}")

    lr = multivariate_logrank_test(
        tmp[time_col],
        tmp[group_col],
        tmp[event_col]
    )

    print(f"\n[{group_col} log-rank]")
    print(lr.test_statistic, lr.p_value)

    return lr


# Stage vs TNM 비교
lr_stage = km_summary_by_group(df_final, "Stage", "Stage")
lr_T     = km_summary_by_group(df_final, "T_stage", "T")
lr_N     = km_summary_by_group(df_final, "N_stage", "N")
lr_M     = km_summary_by_group(df_final, "M_stage", "M")
time_col = "Survival period"
event_col = "Death"
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 9))
axes = axes.flatten()

kmf = KaplanMeierFitter()

plots = [
    ("Stage",    "Stage"),
    ("T_stage", "T stage"),
    ("N_stage", "N stage"),
    ("M_stage", "M stage")
]

for ax, (col, title) in zip(axes, plots):
    tmp = df_final[[time_col, event_col, col]].dropna()

    levels = (
        tmp[col].cat.categories
        if str(tmp[col].dtype) == "category"
        else sorted(tmp[col].unique())
    )

    for lv in levels:
        mask = tmp[col] == lv
        if mask.sum() < 20:
            continue

        kmf.fit(
            durations=tmp.loc[mask, time_col],
            event_observed=tmp.loc[mask, event_col],
            label=str(lv)
        )
        kmf.plot(ax=ax, ci_show=False)

    ax.set_title(title)
    ax.set_xlabel("Time")
    ax.set_ylabel("Survival probability")
    ax.legend(title=col, fontsize=9)

plt.suptitle("Kaplan-Meier Survival Curves: Stage vs TNM", fontsize=14)
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

from lifelines.statistics import logrank_test, multivariate_logrank_test
```

```
time_col = "Survival period"
event_col = "Death"

tmp_stage = df_final[[time_col, event_col, "Stage"]].dropna().copy()
tmp_stage["Stage"] = tmp_stage["Stage"].astype(str)

def stage_3grp(x):
    try:
        xi = int(float(x))
    except:
        return np.nan
    if xi <= 2:
        return "0-2"
    elif xi == 3:
        return "3"
    else:
        return "4"

tmp_stage["Stage_3grp"] = tmp_stage["Stage"].apply(stage_3grp).astype("category")
print("Stage_3grp counts:")
print(tmp_stage["Stage_3grp"].value_counts())

kmf = KaplanMeierFitter()
plt.figure(figsize=(7, 5))

for g in tmp_stage["Stage_3grp"].cat.categories:
    mask = tmp_stage["Stage_3grp"] == g
    if mask.sum() == 0:
        continue
    kmf.fit(
        durations=tmp_stage.loc[mask, time_col],
        event_observed=tmp_stage.loc[mask, event_col],
        label=f"Stage {g}"
    )
    kmf.plot(ci_show=False)

plt.title("KM: Stage regrouped (0-2) vs 3 vs 4")
plt.xlabel("Time")
plt.ylabel("Survival probability")
plt.legend()
plt.tight_layout()
plt.show()

lr_stage3 = multivariate_logrank_test(
```

```
    tmp_stage[time_col],
    tmp_stage["Stage_3grp"],
    tmp_stage[event_col]
)
print("\n[Stage_3grp log-rank]")
print(lr_stage3.test_statistic, lr_stage3.p_value)

tmp_T = df_final[[time_col, event_col, "T_stage"]].dropna().copy()

tmp_T["T_2grp"] = np.where(tmp_T["T_stage"].astype(str) == "T4", "T4", "T1-3")
tmp_T["T_2grp"] = tmp_T["T_2grp"].astype("category")
print("\nT_2grp counts:")
print(tmp_T["T_2grp"].value_counts())

kmf = KaplanMeierFitter()
plt.figure(figsize=(7, 5))

for g in tmp_T["T_2grp"].cat.categories:
    mask = tmp_T["T_2grp"] == g
    if mask.sum() == 0:
        continue
    kmf.fit(
        durations=tmp_T.loc[mask, time_col],
        event_observed=tmp_T.loc[mask, event_col],
        label=f"T {g}"
    )
    kmf.plot(ci_show=False)

plt.title("KM: T_stage regrouped (T1-3) vs T4")
plt.xlabel("Time")
plt.ylabel("Survival probability")
plt.legend()
plt.tight_layout()
plt.show()

mask_A = tmp_T["T_2grp"] == "T1-3"
lr_T2 = logrank_test(
    tmp_T.loc[mask_A, time_col],
    tmp_T.loc[~mask_A, time_col],
    event_observed_A=tmp_T.loc[mask_A, event_col],
    event_observed_B=tmp_T.loc[~mask_A, event_col]
)
print("\n[T_2grp log-rank]")
print(lr_T2.test_statistic, lr_T2.p_value)
```

```
time_col = "Survival period"
event_col = "Death"
exclude_vars = ['Stage', 'T_stage', 'N_stage', 'M_stage']

cat_vars = [
    c for c in df_final.select_dtypes(include='category').columns
    if c not in exclude_vars
]

cat_vars

def km_logrank_summary(df, group_col):
    kmf = KaplanMeierFitter()
    tmp = df[[time_col, event_col, group_col]].dropna().copy()

    print(f"\n=== KM by {group_col} ===")

    for lv in tmp[group_col].cat.categories:
        mask = tmp[group_col] == lv
        if mask.sum() < 10:
            continue

        kmf.fit(
            durations=tmp.loc[mask, time_col],
            event_observed=tmp.loc[mask, event_col],
            label=str(lv)
        )

        print(
            f"{group_col}={lv}: "
            f"n={mask.sum()}, "
            f"median={kmf.median_survival_time_}"
        )

    # log-rank
    lr = multivariate_logrank_test(
        event_durations=tmp[time_col],
        groups=tmp[group_col],
        event_observed=tmp[event_col]
    )

    print(f"[{group_col} log-rank]")
    print(lr.test_statistic, lr.p_value)
```

```
    return {
        "variable": group_col,
        "n_levels": tmp[group_col].nunique(),
        "n": tmp.shape[0],
        "events": tmp[event_col].sum(),
        "chi2": lr.test_statistic,
        "p_value": lr.p_value
    }

rows = []

for v in cat_vars:
    try:
        res = km_logrank_summary(df_final, v)
        rows.append(res)
    except Exception as e:
        print(f"{v}: ERROR → {e}")

# 유의확률 20%로 완화해서 추가 확인

alpha = 0.20

candidates = ["Diagnosis", "Has_birth", "BRCA_status"]

if "km_results" in globals():
    cand_tbl = (
        km_results[km_results["variable"].isin(candidates)]
        .sort_values("p_value")
        .copy()
    )
    cand_tbl["pass_alpha"] = cand_tbl["p_value"] < alpha
    print("=== 후보 변수 요약 (p<0.20 여부 표시) ===")
    print(cand_tbl[["variable", "n_levels", "chi2", "p_value", "pass_alpha"]])

def plot_km(df, group_col, ax, min_n=20):
    kmf = KaplanMeierFitter()
    tmp = df[[time_col, event_col, group_col]].dropna().copy()

    levels = (
        list(tmp[group_col].cat.categories)
        if str(tmp[group_col].dtype) == "category"
        else sorted(tmp[group_col].unique())
    )
```

```
    for lv in levels:
        mask = tmp[group_col] == lv
        if mask.sum() < min_n:
            continue
        kmf.fit(
            durations=tmp.loc[mask, time_col],
            event_observed=tmp.loc[mask, event_col],
            label=str(lv)
        )
        kmf.plot(ax=ax, ci_show=False)

    lr = multivariate_logrank_test(tmp[time_col], tmp[group_col], tmp[event_col])

    ax.set_title(f"KM by {group_col}")
    ax.set_xlabel("Time")
    ax.set_ylabel("Survival probability")
    ax.legend(title=group_col, fontsize=9)

    return {"variable": group_col, "chi2": lr.test_statistic, "p_value": lr.p_value}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

lr_out = []
for ax, v in zip(axes, candidates):
    lr_out.append(plot_km(df_final, v, ax=ax, min_n=20))

plt.tight_layout()
plt.show()

print("\n=== 추가 확인 log-rank 결과 ===")
out_df = pd.DataFrame(lr_out).sort_values("p_value")
out_df["pass_alpha"] = out_df["p_value"] < alpha
print(out_df)

from lifelines import KaplanMeierFitter, NelsonAalenFitter

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# 1) Overall KM survival curve
kmf = KaplanMeierFitter()
kmf.fit(
    durations=df_final[time_col],
    event_observed=df_final[event_col],
```

```
    label="Overall Survival"
)
kmf.plot(ax=axes[0], ci_show=True)
axes[0].set_title("Overall Kaplan-Meier Survival Curve")
axes[0].set_xlabel("Time")
axes[0].set_ylabel("Survival Probability")
axes[0].grid(alpha=0.3)

# 2) Overall cumulative hazard (Nelson-Aalen)
naf = NelsonAalenFitter()
naf.fit(
    durations=df_final[time_col],
    event_observed=df_final[event_col],
    label="Overall Cumulative Hazard"
)
naf.plot(ax=axes[1], ci_show=True)
axes[1].set_title("Overall Cumulative Hazard Function")
axes[1].set_xlabel("Time")
axes[1].set_ylabel("Cumulative Hazard")
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print("Median survival time (KM):", kmf.median_survival_time_)

time_col = "Survival period"
event_col = "Death"

kmf = KaplanMeierFitter()

def plot_km(df, group_col, ax, min_n=20, title=None):
    tmp = df[[time_col, event_col, group_col]].dropna().copy()

    if str(tmp[group_col].dtype) == "category":
        levels = tmp[group_col].cat.categories
    else:
        levels = sorted(tmp[group_col].unique())

    for lv in levels:
        mask = tmp[group_col] == lv
        if mask.sum() < min_n:
            continue
```

```
        kmf.fit(
            durations=tmp.loc[mask, time_col],
            event_observed=tmp.loc[mask, event_col],
            label=f"{lv} (n={mask.sum()})"
        )
        kmf.plot_survival_function(ax=ax, ci_show=False)

    ax.set_title(title if title else f"KM by {group_col}")
    ax.set_xlabel("Time")
    ax.set_ylabel("Survival probability")
    ax.legend(fontsize=8)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 1) T-stage (T1-3 vs T4)
tmp_T = df_final[[time_col, event_col, "T_stage"]].dropna().copy()
tmp_T["T_2grp"] = tmp_T["T_stage"].astype(str).replace(
    {"T1": "T1-3", "T2": "T1-3", "T3": "T1-3", "T4": "T4"}
).astype("category")

plot_km(tmp_T, "T_2grp", axes[0, 0], title="T_stage (T1-3 vs T4)")

# 2) Diagnosis
plot_km(df_final, "Diagnosis", axes[0, 1], title="Diagnosis")

# 3) BRCA_status
plot_km(df_final, "BRCA_status", axes[1, 0], title="BRCA_status")

# 4) Has_birth
plot_km(df_final, "Has_birth", axes[1, 1], title="Has_birth")

plt.tight_layout()
plt.show()

naf = NelsonAalenFitter()

def plot_na(df, group_col, ax, min_n=20, title=None):
    tmp = df[[time_col, event_col, group_col]].dropna().copy()

    if str(tmp[group_col].dtype) == "category":
        levels = tmp[group_col].cat.categories
    else:
        levels = sorted(tmp[group_col].unique())
```

```
    for lv in levels:
        mask = tmp[group_col] == lv
        if mask.sum() < min_n:
            continue

        naf.fit(
            durations=tmp.loc[mask, time_col],
            event_observed=tmp.loc[mask, event_col],
            label=str(lv)
        )
        naf.plot(ax=ax)

    ax.set_title(title if title else f"NA by {group_col}")
    ax.set_xlabel("Time")
    ax.set_ylabel("Cumulative hazard")
    ax.legend(fontsize=8)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 1) T-stage (T1-3 vs T4)
tmp_T = df_final[[time_col, event_col, "T_stage"]].dropna().copy()
tmp_T["T_2grp"] = tmp_T["T_stage"].astype(str).replace(
    {"T1":"T1-3","T2":"T1-3","T3":"T1-3","T4":"T4"}
).astype("category")

plot_na(tmp_T, "T_2grp", axes[0, 0], title="T_stage (T1-3 vs T4)")

# 2) Diagnosis
plot_na(df_final, "Diagnosis", axes[0, 1], title="Diagnosis")

# 3) BRCA_status
plot_na(df_final, "BRCA_status", axes[1, 0], title="BRCA_status")

# 4) Has_birth
plot_na(df_final, "Has_birth", axes[1, 1], title="Has_birth")

plt.tight_layout()
plt.show()

df_u = df_final.copy()
df_u = df_u.rename(columns=lambda c: c.replace(" ", "_"))

time_col_u = time_col.replace(" ", "_")
event_col_u = event_col.replace(" ", "_")
```

```
binary_as_cat = [
    "Has_birth",
    "Menopause_status",
    "M_stage",
    "Chemotherapy",
    "Hormone_therapy",
    "Radiation_Therapy",
]

binary_as_cat = [c for c in binary_as_cat if c in df_u.columns]
for c in binary_as_cat:
    df_u[c] = df_u[c].astype("category")

def univariate_cox(df, var):
    tmp = df[[time_col_u, event_col_u, var]].dropna().copy()

    if tmp[var].nunique(dropna=True) < 2:
        return None

    cph = CoxPHFitter()
    cph.fit(
        tmp,
        duration_col=time_col_u,
        event_col=event_col_u,
        formula=var
    )

    s = cph.summary.reset_index()

    out = {
        "variable": var,
        "variable_display": var.replace("_", " "),
        "type": "categorical" if str(df[var].dtype) == "category" else "continuous",
        "n": tmp.shape[0],
        "events": int(tmp[event_col_u].sum()),
        "c_index": float(cph.concordance_index_),
        "AIC_partial": float(cph.AIC_partial_)
    }

    if s.shape[0] > 1:
        out["n_levels"] = int(tmp[var].nunique())
        out["min_p_value"] = float(s["p"].min())
        out["HR_range"] = f"{s['exp(coef)'].min():.3f}–{s['exp(coef)'].max():.3f}"
    else:
```

```
        out["HR"] = float(s.loc[0, "exp(coef)"])
        out["CI_lower"] = float(s.loc[0, "exp(coef) lower 95%"])
        out["CI_upper"] = float(s.loc[0, "exp(coef) upper 95%"])
        out["p_value"] = float(s.loc[0, "p"])

    return out

covariates = [c for c in df_u.columns if c not in [time_col_u, event_col_u]]

rows = []
errors = []

for v in covariates:
    try:
        res = univariate_cox(df_u, v)
        if res is not None:
            rows.append(res)
    except Exception as e:
        errors.append((v, str(e)))

uni_cox = pd.DataFrame(rows)

uni_cox["p_for_sort"] = uni_cox["p_value"].fillna(uni_cox.get("min_p_value"))
uni_cox = (
    uni_cox.sort_values(["p_for_sort", "c_index"], ascending=[True, False])
           .drop(columns=["p_for_sort"])
           .reset_index(drop=True)
)

print(f"단변량 Cox 완료: {uni_cox.shape[0]}개 변수")
if errors:
    print(f"에러 변수: {len(errors)}개 (상위 10개만 표시)")
    for v, msg in errors[:10]:
        print(f"- {v}: {msg}")

uni_cox

def _extract_level_name(covariate, var):
    s = str(covariate)
    prefix = f"C({var})[T."
    if s.startswith(prefix) and s.endswith("]"):
        return s[len(prefix):-1]
    return s
```

```
def univariate_cox_longtable(df, var, time_col, event_col, var_label=None):
    var_label = var_label or var
    tmp = df[[time_col, event_col, var]].dropna().copy()
    if tmp[var].nunique(dropna=True) < 2:
        return None
    is_cat = (str(tmp[var].dtype) == "category") or (tmp[var].dtype == "object")

    cph = CoxPHFitter()

    if is_cat:
        tmp[var] = tmp[var].astype("category")
        levels = tmp[var].cat.categories.tolist()
        cph.fit(tmp,          duration_col=time_col,          event_col=event_col,
formula=f"C({var})")
        s = cph.summary.reset_index().rename(columns={"index": "covariate"})
        est_levels = [_extract_level_name(cv, var) for cv in s["covariate"].tolist()]
        ref_level = [lv for lv in levels if lv not in est_levels]
        ref_level = ref_level[0] if len(ref_level) == 1 else levels[0]

        rows = []
        rows.append({
            "변수": var_label,
            "수준": f"{ref_level} (ref)",
            "coef": 0.0,
            "HR": 1.0,
            "SE": np.nan,
            "Z": np.nan,
            "p-값": np.nan,
            "신뢰구간 하한": np.nan,
            "신뢰구간 상한": np.nan,
            "n": int(tmp.shape[0]),
            "events": int(tmp[event_col].sum())
        })

        for _, r in s.iterrows():
            lv = _extract_level_name(r["covariate"], var)
            rows.append({
                "변수": var_label,
                "수준": lv,
                "coef": float(r["coef"]),
                "HR": float(r["exp(coef)"]),
                "SE": float(r["se(coef)"]),
                "Z": float(r["z"]),
                "p-값": float(r["p"]),
```

```
                "신뢰구간 하한": float(r["exp(coef) lower 95%"]),
                "신뢰구간 상한": float(r["exp(coef) upper 95%"]),
                "n": int(tmp.shape[0]),
                "events": int(tmp[event_col].sum())
            })

        out = pd.DataFrame(rows)

        order = [f"{ref_level} (ref)"] + [lv for lv in levels if lv != ref_level]
        out["__is_ref"] = out["수준"].str.endswith("(ref)")
        out = out.sort_values(["변수", "__is_ref"], ascending=[True, False]).drop(columns="__is_ref")

    else:
        tmp[var] = pd.to_numeric(tmp[var], errors="coerce")
        tmp = tmp.dropna(subset=[var])

        if tmp[var].nunique(dropna=True) < 2:
            return None

        cph.fit(tmp, duration_col=time_col, event_col=event_col, formula=var)
        r = cph.summary.iloc[0]

        out = pd.DataFrame([{
            "변수": var_label,
            "수준": "-",  # 연속형은 수준 없음
            "coef": float(r["coef"]),
            "HR": float(r["exp(coef)"]),
            "SE": float(r["se(coef)"]),
            "Z": float(r["z"]),
            "p-값": float(r["p"]),
            "신뢰구간 하한": float(r["exp(coef) lower 95%"]),
            "신뢰구간 상한": float(r["exp(coef) upper 95%"]),
            "n": int(tmp.shape[0]),
            "events": int(tmp[event_col].sum())
        }])

    num_cols = ["coef", "HR", "SE", "Z", "p-값", "신뢰구간 하한", "신뢰구간 상한"]
    for c in num_cols:
        out[c] = pd.to_numeric(out[c], errors="coerce").round(4)

    return out

var_list = [
```

```
    ("Diagnosis", "조직학적 진단 (Diagnosis)"),
    ("T_stage", "T 병기 (T-stage)"),
    ("BRCA_status", "BRCA 유전자 상태 (BRCA status)"),
    ("BMI", "BMI (연속형)"),
    # ("treat", "치료 유형 (treat)"),
]

tables = []
for v, lab in var_list:
    if v in df_u.columns:
        t = univariate_cox_longtable(df_u, v, time_col_u, event_col_u,
var_label=lab)
        if t is not None:
            tables.append(t)

table_32 = pd.concat(tables, ignore_index=True)

table_32 = table_32[[
    "변수", "수준", "coef", "HR", "SE", "Z", "p-값", "신뢰구간 하한", "신뢰구간 상한"
]]

table_32

time_col = "Survival period"
event_col = "Death"

df_null = df_final[[time_col, event_col]].copy()
df_null = df_null[df_null[time_col] > 0]

cph_null = CoxPHFitter()
cph_null.fit(df_null, duration_col=time_col, event_col=event_col)

print("Null Cox n:", df_null.shape[0], "events:", int(df_null[event_col].sum()))
print("Null Cox C-index:", cph_null.concordance_index_)
print("Null Cox AIC_partial:", cph_null.AIC_partial_)

cox_vars = [
    "Stage",
    "AGE", "BMI",
    "Has_birth", "Menopause_status",
    "Diagnosis",
    "ER_clean", "PR_clean", "AR_clean",
    "BRCA_status",
    "Smoke",
```

```
    "treat",
]
cox_vars = [v for v in cox_vars if v in df_final.columns]

df_cox = df_final[[time_col, event_col] + cox_vars].copy()
df_cox = df_cox.dropna()

print("Cox sample size:", df_cox.shape[0])
print("Events:", int(df_cox[event_col].sum()))

cat_for_cox = [
    "Stage", "Diagnosis",
    "ER_clean", "PR_clean", "AR_clean",
    "BRCA_status", "treat",
    "Has_birth", "Menopause_status", "Smoke",
]
cat_for_cox = [c for c in cat_for_cox if c in df_cox.columns]

cont_for_cox = [c for c in cox_vars if c not in cat_for_cox]

formula_terms = [f"C({c})" for c in cat_for_cox] + cont_for_cox
formula = " + ".join(formula_terms)

print("\n[formula]")
print(formula)

cph_full = CoxPHFitter()
cph_full.fit(df_cox, duration_col=time_col, event_col=event_col, formula=formula)

cph_full.print_summary()
print("\nFull Cox C-index:", cph_full.concordance_index_)
print("Full Cox AIC_partial:", cph_full.AIC_partial_)

def backward_elimination_aic(
    df,
    time_col="Survival period",
    event_col="Death",
    covariates=None,
    categorical=None,
    penalizer=0.0,
    l1_ratio=0.0,
    min_vars=1,
    max_steps=200,
    verbose=True
```

```
):
    Returns
    -------
    result : dict
        {
          "selected": 최종 공변량 리스트,
          "history": 단계별 로그(DataFrame),
          "final_model": 최종 CoxPHFitter,
          "final_formula": 최종 formula 문자열,
          "final_aic": 최종 AIC_partial,
          "final_cindex": 최종 concordance_index_,
          "n": 표본수,
          "events": 사건수
        }
    """
    if covariates is None:
        covariates = [c for c in df.columns if c not in [time_col, event_col]]

    covariates = [c for c in covariates if c in df.columns]
    if categorical is None:
        categorical = []

    categorical = [c for c in categorical if c in covariates]

    def build_formula(vars_):
        cat = [v for v in vars_ if v in categorical]
        cont = [v for v in vars_ if v not in categorical]
        terms = [f"C({v})" for v in cat] + cont
        return " + ".join(terms) if terms else "1"

    use_cols = [time_col, event_col] + covariates
    df_work = df[use_cols].copy().dropna()
    n = df_work.shape[0]
    events = int(df_work[event_col].sum())

    if verbose:
        print(f"[Backward AIC] n={n}, events={events}, start_vars={len(covariates)}")

    current_vars = covariates.copy()
    current_formula = build_formula(current_vars)

    cph = CoxPHFitter(penalizer=penalizer, l1_ratio=l1_ratio)
    cph.fit(df_work,          duration_col=time_col,          event_col=event_col,
formula=current_formula)
```

```
    current_aic = float(cph.AIC_partial_)
    current_cindex = float(cph.concordance_index_)

    history = [{
        "step": 0,
        "action": "start",
        "dropped": None,
        "n_vars": len(current_vars),
        "AIC_partial": current_aic,
        "c_index": current_cindex,
        "formula": current_formula
    }]

    step = 0
    while step < max_steps and len(current_vars) > min_vars:
        step += 1

        best_aic = current_aic
        best_drop = None
        best_model = None
        best_formula = None
        best_cindex = None

        for drop_var in current_vars:
            trial_vars = [v for v in current_vars if v != drop_var]
            trial_formula = build_formula(trial_vars)

            try:
                m = CoxPHFitter(penalizer=penalizer, l1_ratio=l1_ratio)
                m.fit(df_work,      duration_col=time_col,      event_col=event_col,
formula=trial_formula)
                aic = float(m.AIC_partial_)
                if aic < best_aic - 1e-8:
                    best_aic = aic
                    best_drop = drop_var
                    best_model = m
                    best_formula = trial_formula
                    best_cindex = float(m.concordance_index_)
            except Exception:
                continue

        if best_drop is None:
            if verbose:
                print(f"[Stop] no AIC improvement at step={step}.")
```

```
            break

        current_vars.remove(best_drop)
        current_aic = best_aic
        current_formula = best_formula
        cph = best_model
        current_cindex = best_cindex

        history.append({
            "step": step,
            "action": "drop",
            "dropped": best_drop,
            "n_vars": len(current_vars),
            "AIC_partial": current_aic,
            "c_index": current_cindex,
            "formula": current_formula
        })

        if verbose:
            print(f"[Step {step}] drop={best_drop} -> AIC={current_aic:.2f},
c-index={current_cindex:.3f}, n_vars={len(current_vars)}")

    history_df = pd.DataFrame(history)

    return {
        "selected": current_vars,
        "history": history_df,
        "final_model": cph,
        "final_formula": current_formula,
        "final_aic": current_aic,
        "final_cindex": current_cindex,
        "n": n,
        "events": events
    }

time_col = "Survival period"
event_col = "Death"

cox_vars_tnm = [
    "T_stage", "N_stage", "M_stage",
    "AGE", "BMI",
    "Has_birth", "Menopause_status",
    "Diagnosis",
    "ER_clean", "PR_clean", "AR_clean",
```

```
    "BRCA_status",
    "Smoke",
    "treat"
]
cox_vars_tnm = [v for v in cox_vars_tnm if v in df_final.columns]

categorical_tnm = [
    "T_stage", "N_stage", "M_stage",
    "Diagnosis",
    "ER_clean", "PR_clean", "AR_clean",
    "BRCA_status",
    "treat",
    "Has_birth", "Menopause_status", "Smoke",
]
categorical_tnm = [c for c in categorical_tnm if c in df_final.columns]

res = backward_elimination_aic(
    df_final,
    time_col=time_col,
    event_col=event_col,
    covariates=cox_vars_tnm,
    categorical=categorical_tnm,
    penalizer=0.0,
    l1_ratio=0.0,
    min_vars=1,
    verbose=True
)

print("\n=== 최종 선택 변수 ===")
print(res["selected"])

print("\n=== 제거 히스토리(상위 10줄) ===")
print(res["history"].head(10))

print("\n=== 최종 모형 요약 ===")
print("Final formula:", res["final_formula"])
print("Final C-index:", res["final_cindex"])
print("Final AIC_partial:", res["final_aic"])

final_cat = ["BRCA_status", "Diagnosis"]
final_num = ["BMI"]

use_cols = [time_col, event_col] + final_cat + final_num
use_cols = [c for c in use_cols if c in df_final.columns]
```

```
df_cox = df_final[use_cols].copy()

for c in final_cat:
    if c in df_cox.columns:
        df_cox[c] = df_cox[c].astype("category")

for c in final_num:
    if c in df_cox.columns:
        df_cox[c] = pd.to_numeric(df_cox[c], errors="coerce")

df_cox[time_col] = pd.to_numeric(df_cox[time_col], errors="coerce")
df_cox[event_col] = pd.to_numeric(df_cox[event_col], errors="coerce").astype(int)

df_cox = df_cox.dropna()

print("Final Cox sample size:", df_cox.shape[0])
print("Events:", int(df_cox[event_col].sum()))

formula_terms = []
for c in final_cat:
    if c in df_cox.columns:
        formula_terms.append(f"C({c})")
for c in final_num:
    if c in df_cox.columns:
        formula_terms.append(c)

final_formula = " + ".join(formula_terms)
print("\n[final formula]")
print(final_formula)

cph_final = CoxPHFitter()
cph_final.fit(
    df_cox,
    duration_col=time_col,
    event_col=event_col,
    formula=final_formula
)

cph_final.print_summary()

print("\nFinal Cox C-index:", cph_final.concordance_index_)
print("Final Cox AIC_partial:", cph_final.AIC_partial_)
```

```
from scipy.stats import chi2

need_cols = [time_col, event_col, "BRCA_status", "Diagnosis", "BMI"]
df_int = df_final[need_cols].copy()

df_int = df_int.dropna().reset_index(drop=True)

print("Interaction test sample size:", df_int.shape[0], "events:",
int(df_int[event_col].sum()))

base_formula = "C(BRCA_status) + C(Diagnosis) + BMI"

cph_base = CoxPHFitter()
cph_base.fit(df_int, duration_col=time_col, event_col=event_col,
formula=base_formula)

LL_base = cph_base.log_likelihood_
AIC_base = cph_base.AIC_partial_
C_base = cph_base.concordance_index_

print("\n[Base model]")
print("formula:", base_formula)
print("logL:", LL_base, "AIC_partial:", AIC_base, "C-index:", C_base)

tests = [
    ("BRCA x Diagnosis", "C(BRCA_status):C(Diagnosis)"),
    ("BMI x Diagnosis",  "BMI:C(Diagnosis)"),
    ("BMI x BRCA",       "BMI:C(BRCA_status)"),
]

rows = []
for name, inter_term in tests:
    formula = base_formula + " + " + inter_term

    cph = CoxPHFitter()
    cph.fit(df_int, duration_col=time_col, event_col=event_col, formula=formula)

    LL = cph.log_likelihood_
    AIC = cph.AIC_partial_
    C = cph.concordance_index_

    LR = 2 * (LL - LL_base)
    df_diff = cph.params_.shape[0] - cph_base.params_.shape[0]
    p_lrt = 1 - chi2.cdf(LR, df=df_diff)
```

```
    s = cph.summary.copy()
    inter_rows = s.loc[s.index.str.contains(":", regex=False), ["coef", "exp(coef)", "p"]].copy()

    min_p = inter_rows["p"].min() if len(inter_rows) > 0 else np.nan
    sig_cnt_005 = int((inter_rows["p"] < 0.05).sum()) if len(inter_rows) > 0 else 0
    sig_cnt_010 = int((inter_rows["p"] < 0.10).sum()) if len(inter_rows) > 0 else 0

    rows.append({
        "test": name,
        "formula_added": inter_term,
        "n_params_base": int(cph_base.params_.shape[0]),
        "n_params_full": int(cph.params_.shape[0]),
        "df_diff": int(df_diff),
        "LR_stat": float(LR),
        "LRT_pvalue": float(p_lrt),
        "AIC_base": float(AIC_base),
        "AIC_with_inter": float(AIC),
        "AIC_delta": float(AIC - AIC_base),
        "C_base": float(C_base),
        "C_with_inter": float(C),
        "C_delta": float(C - C_base),
        "min_p_inter_terms": float(min_p) if pd.notna(min_p) else np.nan,
        "n_sig_inter_p<0.05": sig_cnt_005,
        "n_sig_inter_p<0.10": sig_cnt_010,
    })

    print(f"\n[{name}]")
    print("added:", inter_term)
    print("LRT: LR=", LR, "df=", df_diff, "p=", p_lrt)
    print("AIC delta:", AIC - AIC_base, "(<0이면 좋아짐)")
    print("C-index delta:", C - C_base)
    if len(inter_rows) > 0:
        print("interaction terms (top 10):")
        print(inter_rows.sort_values("p").head(10))
    else:
        print("No interaction rows found in summary (check term string).")

res = pd.DataFrame(rows).sort_values(["LRT_pvalue", "AIC_delta"], ascending=[True, True])
print("\n=== Interaction test summary ===")
display(res)
```

```
time_col = "Survival period"
event_col = "Death"
final_formula = "C(BRCA_status) + C(Diagnosis) + BMI"

cox_cols = [time_col, event_col, "BRCA_status", "Diagnosis", "BMI"]
df_cox = df_final[cox_cols].dropna().copy()

print("Final Cox sample size:", df_cox.shape[0])
print("Events:", int(df_cox[event_col].sum()))

cph_final = CoxPHFitter()
cph_final.fit(
    df_cox,
    duration_col=time_col,
    event_col=event_col,
    formula=final_formula
)

print("C-index:", cph_final.concordance_index_)
print("AIC_partial:", cph_final.AIC_partial_)

cph_final.check_assumptions(
    df_cox,
    p_value_threshold=0.05,
    show_plots=True
)

ph = cph_final.predict_partial_hazard(df_cox).astype(float)
base_ch = cph_final.baseline_cumulative_hazard_.iloc[:, 0]   # Series
timeline = base_ch.index.values.astype(float)
Hvals = base_ch.values.astype(float)

T = df_cox[time_col].astype(float).values
E = df_cox[event_col].astype(int).values

idx = np.searchsorted(timeline, T, side="right") - 1
idx = np.clip(idx, 0, len(timeline) - 1)
H0_T = Hvals[idx]

mart = E - H0_T * ph.values

eps = 1e-12
term = mart + E * np.log(np.clip(E - mart, eps, None))
dev = np.sign(mart) * np.sqrt(np.clip(-2.0 * term, 0.0, None))
```

```
resid = df_cox.copy()
resid["martingale"] = mart
resid["deviance"] = dev

print(resid[["martingale", "deviance"]].describe())

def ph_plots_for_categorical(
    df, group_col,
    time_col="Survival period", event_col="Death",
    min_n=30,
    eps=1e-9,
    title_prefix=None
):
    tmp = df[[time_col, event_col, group_col]].dropna().copy()

    counts = tmp[group_col].value_counts()
    levels = counts[counts >= min_n].index.tolist()

    if len(levels) == 0:
        print(f"[{group_col}] min_n={min_n} 이상 그룹이 없습니다.")
        return

    kmf = KaplanMeierFitter()
    naf = NelsonAalenFitter()

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    axS, axLL, axH, axlogH = axes[0,0], axes[0,1], axes[1,0], axes[1,1]

    for lv in levels:
        mask = (tmp[group_col] == lv)
        d = tmp.loc[mask, time_col].astype(float)
        e = tmp.loc[mask, event_col].astype(int)

        # 1) KM survival
        kmf.fit(durations=d, event_observed=e, label=str(lv))
        sf = kmf.survival_function_.copy()
        axS.plot(sf.index.values, sf[kmf._label].values, drawstyle="steps-post",
                 label=f"{lv} (n={mask.sum()})")

        # 2) log(-log(S(t))) vs log(t)
        t = sf.index.values
        S = np.clip(sf[kmf._label].values, eps, 1 - eps)
        x = np.log(np.clip(t, eps, None))
```

```
        y = np.log(-np.log(S))
        axLL.plot(x, y, label=str(lv))

        # 3) Nelson-Aalen cumulative hazard
        naf.fit(durations=d, event_observed=e, label=str(lv))
        ch = naf.cumulative_hazard_.copy()
        axH.plot(ch.index.values,      ch[str(lv)].values,      drawstyle="steps-post",
label=str(lv))

        # 4) log(H(t)) vs log(t)
        tt = ch.index.values
        H = np.clip(ch[str(lv)].values, eps, None)
        xx = np.log(np.clip(tt, eps, None))
        yy = np.log(H)
        axlogH.plot(xx, yy, label=str(lv))

    main_title = f"{group_col} PH check"
    if title_prefix:
        main_title = f"{title_prefix}: {main_title}"
    fig.suptitle(main_title, fontsize=14)

    axS.set_title("KM survival S(t)")
    axS.set_xlabel("Time")
    axS.set_ylabel("Survival probability")
    axS.legend(fontsize=8)

    axLL.set_title("log(-log(S(t))) vs log(Time)  (PH check)")
    axLL.set_xlabel("log(Time)")
    axLL.set_ylabel("log(-log(S(t)))")
    axLL.legend(fontsize=8)

    axH.set_title("Nelson-Aalen cumulative hazard H(t)")
    axH.set_xlabel("Time")
    axH.set_ylabel("Cumulative hazard")
    axH.legend(fontsize=8)

    axlogH.set_title("log(H(t)) vs log(Time)  (PH check)")
    axlogH.set_xlabel("log(Time)")
    axlogH.set_ylabel("log(Cumulative hazard)")
    axlogH.legend(fontsize=8)

    plt.tight_layout()
    plt.show()
```

```
        print(f"\n[{group_col}] counts (min_n={min_n} applied)")
        print(counts.loc[levels])

    for col in ["Diagnosis", "BRCA_status"]:
        ph_plots_for_categorical(df_cox, col, title_prefix="Final model")
```